

HELM

Comprehensive Study Notes

CKA / CKAD / KSNA Exam Preparation

Zero to Production-Expert Level

Covers: Charts · Releases · Templates · Repositories

Hooks · Security · CI/CD · Troubleshooting · 95 Q&A

Helm v3 — Production Ready

PART 1: FOUNDATIONS

Chapter 1 — What is Helm and Why Does It Exist?

The Problem Before Helm

Imagine you have a microservices application with 10 components: a frontend, backend API, database, cache, message queue, and supporting services. Each component needs a Kubernetes **Deployment**, a **Service**, possibly an **Ingress**, **ConfigMaps**, **Secrets**, **HorizontalPodAutoscalers**, and **PodDisruptionBudgets**. That easily becomes 60–80 YAML files.

Managing these raw YAML files at scale creates severe problems:

- **Duplication:** Image tags, resource limits, labels, and namespace names are repeated across dozens of files. Changing the image tag for a deployment means editing multiple files.
- **No versioning:** There is no built-in concept of 'version 1.2.3 of my app deployment'. You can't say 'roll back to last week's configuration' without custom tooling.
- **No lifecycle management:** kubectl apply deploys resources but doesn't know which resources belong together. Deleting an application means manually tracking and deleting 60+ resources.
- **No templating:** Deploying the same application to dev, staging, and prod means maintaining near-identical YAML files with only a few values changed (namespace, replica count, image tag, resource limits).
- **No dependency management:** If your app depends on a database chart, there is no built-in way to declare and install that dependency.
- **No release tracking:** If you kubectl apply -f . on Monday and a colleague does the same on Wednesday with different files, you have no audit trail of what changed.

What Is a Package Manager in the Kubernetes Context?

A **package manager** is a tool that automates the installation, upgrading, configuration, and removal of software packages. The analogy to other ecosystems makes this concrete:

Package Manager	Ecosystem	What It Manages
apt / yum / dnf	Linux OS	System packages (.deb, .rpm)
npm / yarn	Node.js	JavaScript packages (node_modules)
pip / poetry	Python	Python packages (site-packages)
brew	macOS	macOS software packages
helm	Kubernetes	Kubernetes application packages (Charts)

Just as apt install nginx downloads nginx, resolves dependencies, configures it, and registers it so you can later run apt remove nginx to cleanly uninstall it, helm install my-nginx bitnami/nginx downloads a Helm chart, renders all required Kubernetes manifests, applies them to the cluster, and registers the release so you can later run helm uninstall my-nginx to cleanly remove everything.

Helm as the Missing Layer

Helm sits between raw **kubectl** and full GitOps/Operator patterns. It is the 'missing layer' that provides:

- **Templating:** Parameterized YAML templates so one chart can deploy to dev with 1 replica and prod with 5 replicas by changing a single value.
- **Packaging:** An entire application (all its K8s resources) bundled into a single .tgz archive called a Chart.
- **Release management:** Named, versioned installations called Releases with full history, upgrade, and rollback capabilities.
- **Dependency management:** Charts can declare other charts as dependencies (e.g., your app chart declares a dependency on the bitnami/postgresql chart).
- **Repository ecosystem:** A global ecosystem of pre-built charts for common software (databases, ingress controllers, monitoring stacks) hosted on Artifact Hub.

The Three Core Concepts

1. Chart — The Package

A **Chart** is a collection of files that describe a set of Kubernetes resources. Think of it as the recipe or blueprint. It contains Go template files, default values, and metadata. A chart is reusable — the same chart can create many different installations. Charts are stored as .tgz archives or in directories.

2. Release — The Instance

A **Release** is a specific instance of a chart deployed to a Kubernetes cluster with a specific set of values. You can install the same chart multiple times, creating multiple releases. Each release has a unique name within a namespace. Example: you might install the bitnami/mysql chart twice — once as prod-db and once as staging-db — each with different passwords and resource limits.

3. Repository — The Registry

A **Repository** is an HTTP server hosting a collection of charts. It serves an index.yaml file that lists all available charts and their versions. Helm has a global public registry called **Artifact Hub** (artifacthub.io) that indexes charts from hundreds of repositories. You can also host private repositories.

■ **NOTE:** The relationship: A **Chart** is like a class definition. A **Release** is like an instance of that class. A **Repository** is like a package registry (npm, PyPI) where you find charts to install.

Helm v2 vs Helm v3 — The Tiller Revolution

Helm v2 had a server-side component called **Tiller** that ran inside the Kubernetes cluster. Helm v3 (released November 2019) removed Tiller entirely. Understanding why is critical for exam questions about security.

Aspect	Helm v2 (with Tiller)	Helm v3 (no Tiller)
Architecture	Client (helm CLI) + Server (Tiller pod in cluster)	Client only (helm CLI)
Communication	helm CLI → Tiller (gRPC) → K8s API	helm CLI → K8s API directly
Security	Tiller ran with cluster-admin by default — full cluster access	Uses user's kubeconfig credentials — fine-grained RBAC
Release storage	ConfigMaps in kube-system namespace	Secrets in the release namespace (default)
Multi-tenancy	Difficult — Tiller had global permissions	Native — each namespace isolated
3-way merge	No — 2-way merge only	Yes — compares old manifest, new manifest, and live state
CRD handling	Limited	First-class CRD support with crds/ directory
Status	Deprecated, EOL	Current, actively maintained

■ **COMMON MISTAKE:** The biggest Tiller security risk: A Tiller deployed with cluster-admin permissions meant that anyone who could talk to Tiller could perform any action in the entire cluster — even if the user's own kubeconfig only had limited permissions. This completely bypassed Kubernetes RBAC. Helm v3 fixes this by making helm use the user's own credentials directly.

The 3-Way Strategic Merge (Helm v3)

Helm v3 introduced **3-way strategic merge** for upgrades. When you run `helm upgrade`, Helm compares three states:

- **Old manifest:** What Helm deployed last time (stored in the release secret)
- **New manifest:** What the new chart renders to
- **Live state:** What is currently running in the cluster (may have been manually edited)

This allows Helm to detect and preserve manual changes made directly with `kubectl`, rather than blindly overwriting them. Helm v2's 2-way merge only compared old and new manifests, ignoring live changes.

When NOT to Use Helm

Helm is powerful but not always the right tool. Use alternatives in these scenarios:

Scenario	Better Tool	Reason
Simple config overlays without templating	Kustomize	No templating language needed — just patch YAML
One-off <code>kubectl</code> commands / quick fixes	<code>kubectl</code>	No overhead of chart structure

Scenario	Better Tool	Reason
Complex stateful applications with deep domain logic	Operator Pattern (Kubebuilder, Operator SDK)	Custom controllers with reconciliation loops for complex state
Config management across many clusters with drift detection	ArgoCD / Flux (GitOps)	Continuous reconciliation, not just one-time deploy
Policy enforcement and admission control	Kyverno / OPA Gatekeeper	These are admission webhooks, not package managers

■ **NOTE:** Helm and Kustomize are complementary, not competing. ArgoCD and Flux both support Helm natively — they run helm template internally and apply the rendered manifests. In production GitOps, you often use Helm for packaging AND ArgoCD for delivery.

Chapter Summary

- ✓ Helm solves YAML duplication, versioning, lifecycle management, and dependency management at scale.
- ✓ The three core concepts are: Chart (package), Release (instance), Repository (registry).
- ✓ Helm v3 removed Tiller — no server-side component, uses user's RBAC credentials directly.
- ✓ 3-way strategic merge in v3 preserves manual live-state changes during upgrades.
- ✓ Alternatives: Kustomize (simple overlays), Operators (complex stateful apps), GitOps (continuous reconciliation).

Chapter 2 — Architecture Deep Dive

Helm Client Architecture (v3)

In Helm v3, the **helm** binary is the only component — there is no server-side component. When you run a **helm** command:

- **Step 1:** helm reads your `$KUBECONFIG` (or `~/.kube/config`) to determine which cluster and namespace to target.
- **Step 2:** helm loads the chart (from local directory, .tgz file, or repository cache).
- **Step 3:** helm renders Go templates using the provided values to produce plain Kubernetes YAML manifests.
- **Step 4:** helm communicates directly with the Kubernetes API server using the same credentials as `kubectl` — no proxy, no Tiller.
- **Step 5:** helm applies the rendered manifests to the cluster via the API server.
- **Step 6:** helm stores a release record (encoded manifest, values, metadata) as a Kubernetes Secret in the release's namespace.

Release Storage Mechanism

Helm must persist release state so it can perform upgrades, rollbacks, and history queries. The default backend is Kubernetes **Secrets**. The storage backend is controlled by the `HELM_DRIVER` environment variable:

Driver	Storage Backend	Default?	Use Case
secret	Kubernetes Secrets (base64 gzip)	YES (v3)	Production — encrypted at rest if etcd is encrypted
configmap	Kubernetes ConfigMaps	No	Debugging — human-readable but not encrypted
memory	In-process memory only	No	Testing — data lost when helm process exits
sql	PostgreSQL/SQLite database	No	Large clusters with many releases — better query performance

Each release revision is stored as a separate Secret. The Secret name follows the pattern `sh.helm.release.v1.<release-name>.v<revision>`. The Secret's `release` data field contains a base64-encoded, gzip-compressed JSON object with the full release metadata including the rendered manifest.

```
# Find all Helm release secrets in a namespace
kubectl get secrets -n my-namespace --field-selector type=helm.sh/release.v1

# Decode a specific release secret (revision 3)
kubectl get secret sh.helm.release.v1.myapp.v3 -n my-namespace \
  -o jsonpath='{.data.release}' | base64 -d | base64 -d | gunzip | python3 -m json.tool
```

■ **NOTE:** The release Secret is base64-encoded TWICE and then gzip-compressed. The first base64 is the Kubernetes Secret data encoding; the second is Helm's own encoding of the JSON payload. This is why you need to pipe through base64 -d twice.

\$KUBECONFIG and Cluster Context

Helm fully inherits kubectl's cluster configuration system. The connection target is determined by this priority order:

- **--kube-context flag:** Explicitly specify a context name: `helm install ... --kube-context prod-cluster`
- **\$KUBECONFIG env var:** Path(s) to kubeconfig files. Multiple files can be colon-separated.
- **Default file:** `~/.kube/config`

```
# Deploy to a specific cluster context
helm install myapp ./mychart --kube-context production

# Use a different kubeconfig file
KUBECONFIG=/path/to/prod-kubeconfig.yaml helm install myapp ./mychart

# Check which context helm will use
kubectl config current-context
```

Helm Storage Paths — Cache, Config, Data

Helm uses three local directory paths. On Linux (following XDG Base Directory Specification):

Path Variable	Default on Linux	What It Stores
\$HELM_CACHE_HOME	~/.cache/helm	Downloaded chart archives (.tgz), repository index files (index.yaml)
\$HELM_CONFIG_HOME	~/.config/helm	Repository configuration (repositories.yaml), registry credentials
\$HELM_DATA_HOME	~/.local/share/helm	Plugins, starters (chart templates)

On macOS these default to `~/Library/...` and on Windows to `%APPDATA%\helm\`.

HELM_* Environment Variables Reference

Variable	Description	Practical Use
HELM_CACHE_HOME	Override chart cache directory	CI: point to a shared cache
HELM_CONFIG_HOME	Override config directory	CI: isolate credentials per pipeline
HELM_DATA_HOME	Override data directory	Custom plugin storage location
HELM_DEBUG	Set to '1' to enable verbose debug output	Debugging template rendering issues
HELM_DRIVER	Release storage backend: secret, configmap, memory, sql	Use 'configmap' for debugging
HELM_KUBEAPISERVER	Override Kubernetes API server URL	Direct cluster access without kubeconfig
HELM_KUBECAFILE	Path to CA cert file for API server TLS	Mutual TLS to API server
HELM_KUBECERTFILE	Path to client cert for API server	Mutual TLS client cert
HELM_KUBEKEYFILE	Path to client key for API server	Mutual TLS client key
HELM_KUBECONTEXT	Override kubeconfig context	Target a specific cluster without --kube-context flag
HELM_KUBETOKEN	Bearer token for API server auth	ServiceAccount token auth in CI/CD
HELM_KUBEINSECURE_SKIP_TLS_VERIFY	Skip TLS verify (insecure!)	Dev only — never production
HELM_NAMESPACE	Default namespace	Avoid typing -n on every command
HELM_MAX_HISTORY	Max revision history per release (default: 10)	Limit Secret count in busy namespaces
HELM_PLUGINS	Override plugins directory path	Custom plugin location in CI
HELM_REGISTRY_CONFIG	OCI registry auth config path	Private OCI registry credentials
HELM_REPOSITORY_CACHE	Override chart repository cache path	Shared cache in CI systems
HELM_REPOSITORY_CONFIG	Override repository list file path	Per-project repo configuration

■ **EXAM TIP:** In CI/CD pipelines, set HELM_KUBETOKEN, HELM_KUBEAPISERVER, and HELM_KUBECAFILE instead of mounting a kubeconfig file. This is the recommended approach for GitLab CI, GitHub Actions, and Jenkins when using ServiceAccount-based authentication.

Chapter Summary

- ✓ Helm v3 is client-only — no Tiller, no server-side component.
- ✓ Helm communicates directly with the K8s API server using your kubeconfig credentials.
- ✓ Release state is stored as Kubernetes Secrets by default (base64+gzip encoded).
- ✓ Three local paths: Cache (charts), Config (repos/creds), Data (plugins).
- ✓ HELM_* env vars override every aspect of Helm's behavior — critical for CI/CD.

Chapter 3 — Installation & Environment Setup

Installation Methods

Method	Command	Pros	Cons
Snap (Linux)	sudo snap install helm --classic	Auto-updates, easy	Snap confinement issues on some distros
APT (Debian/Ubuntu)	curl https://baltocdn.com/helm/signing.asc gpg --dearmor sudo tee /usr/share/keyrings/helm.gpg > /dev/null sudo apt-get install helm	System package manager, pinnable version	Extra setup steps
Homebrew (macOS)	brew install helm	Easy on macOS, version pinnable	macOS only
Binary (all platforms)	Download from GitHub Releases, extract, move to /usr/local/bin/	Exact version control, air-gapped	Manual updates
Install script	curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 bash	One-liner, any platform	Runs remote code — verify signature first

Snap Installation (Your Environment)

```
# Install helm via snap (Kubuntu / Ubuntu)
sudo snap install helm --classic

# Verify installation
helm version
# Output: version.BuildInfo{Version:"v3.x.x", ...}

# See all helm environment variables
helm env
```

Sample helm env Output Explained

```
$ helm env
HELM_BIN="/snap/bin/helm"
HELM_BURST_LIMIT="100"
HELM_CACHE_HOME="/home/user/.cache/helm"
HELM_CONFIG_HOME="/home/user/.config/helm"
HELM_DATA_HOME="/home/user/.local/share/helm"
HELM_DEBUG="false"
HELM_KUBEAPISERVER=""
HELM_KUBECONTEXT=""
HELM_KUBENS=""          # Current namespace from kubeconfig
HELM_MAX_HISTORY="10"
HELM_NAMESPACE="default"
HELM_PLUGINS="/home/user/.local/share/helm/plugins"
HELM_REGISTRY_CONFIG="/home/user/.config/helm/registry/config.json"
HELM_REPOSITORY_CACHE="/home/user/.cache/helm/repository"
HELM_REPOSITORY_CONFIG="/home/user/.config/helm/repositories.yaml"
HELM_VERSION="v3.x.x"  # Helm version
```

Shell Autocompletion

```
# Bash autocompletion (add to ~/.bashrc)
source <(helm completion bash)
echo "source <(helm completion bash)" >> ~/.bashrc

# Zsh autocompletion (add to ~/.zshrc)
source <(helm completion zsh)
echo "source <(helm completion zsh)" >> ~/.zshrc

# Fish
helm completion fish | source

# Reload shell
source ~/.bashrc
```

Helm Plugins

Helm plugins extend the CLI with additional subcommands. They are installed from Git repositories or local directories.

```
# Install a plugin
helm plugin install https://github.com/databus23/helm-diff

# List installed plugins
helm plugin list

# Update a plugin
helm plugin update diff

# Uninstall a plugin
helm plugin uninstall diff

# Essential production plugins:
# helm-diff:    Shows diff between current and proposed changes
# helm-secrets: Encrypts values files with Mozilla SOPS / Vault
# helm-unittest: Unit testing for Helm chart templates
# helm-docs:    Auto-generate documentation from chart values
```

■ **EXAM TIP:** For CKA/CKAD exams: you won't need to install plugins during the exam, but you should understand what they do conceptually, especially helm-diff and helm-secrets. The exam focuses on core helm commands.

Chapter Summary

- ✓ snap install helm --classic is the easiest method on Kubuntu/Ubuntu.
- ✓ helm version and helm env verify installation and show all configuration.
- ✓ Shell autocompletion via 'source <(helm completion bash)' speeds up exam work.
- ✓ Plugins extend Helm — helm-diff and helm-secrets are essential for production.

PART 2: CORE CONCEPTS IN DEPTH

Chapter 4 — Charts: Anatomy and Structure

What Is a Chart?

A **Chart** is a collection of files that describe a set of Kubernetes resources. It contains Go template files that are rendered using values to produce valid Kubernetes YAML manifests. A chart is reusable — the same chart can be deployed multiple times with different values to create different releases.

```
# Create a new chart scaffold
helm create nginx-deploy-chart

# Resulting directory structure:
nginx-deploy-chart/
├── Chart.yaml           # Chart metadata (name, version, description)
├── values.yaml          # Default configuration values
├── charts/              # Sub-chart dependencies
├── templates/           # Go template files → rendered to K8s manifests
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── serviceaccount.yaml
│   ├── hpa.yaml
│   ├── _helpers.tpl     # Named templates / partials (not rendered directly)
│   ├── NOTES.txt        # Post-install instructions (not a K8s manifest)
│   └── tests/
│       └── test-connection.yaml
└── .helmignore          # Files to exclude from chart package
```

Chart.yaml — Complete Field Reference

```
# Chart.yaml – all fields explained
apiVersion: v2           # REQUIRED. 'v2' for Helm v3. 'v1' is Helm v2 only.
name: nginx-deploy-chart # REQUIRED. Chart name. Must match directory name.
version: "1.2.3"         # REQUIRED. Chart version (SemVer 2). Incremented on chart changes.
appVersion: "1.25.0"     # OPTIONAL. Version of the app inside the chart. Informational only.
description: A Helm chart for deploying nginx # OPTIONAL but recommended.
type: application        # OPTIONAL. 'application' (default) or 'library'.
                        # Library charts contain only named templates, no deployable resources.

# Optional metadata fields
keywords:
  - nginx
  - web
  - proxy
home: https://nginx.org # Project homepage URL
sources:
  - https://github.com/my-org/nginx-deploy-chart
maintainers:
```

```
- name: Safdar Ali Shah
  email: safdar@example.com
  url: https://linkedin.com/in/kubernetes

# Dependency declarations
dependencies:
  - name: postgresql          # Chart name in repository
    version: "12.x.x"         # Version constraint (SemVer range)
    repository: "https://charts.bitnami.com/bitnami"
    condition: postgresql.enabled # values.yaml key to enable/disable
    tags:
      - database
    alias: db                  # Override the sub-chart's release name

# Annotations (arbitrary key-value metadata)
annotations:
  "artifacthub.io/category": "database"
  "artifacthub.io/license": "Apache-2.0"
```

version vs appVersion: This distinction is critical. `version` is the chart package version — increment it whenever you change templates, values structure, or chart logic. `appVersion` is purely informational — it documents what application version the chart is designed for, but Helm doesn't use it for anything. The actual image tag is controlled by `values.yaml`.

■ **EXAM TIP:** EXAM TIP: `version` follows SemVer 2 and is the chart version. `appVersion` is the application version (e.g., `nginx:1.25.0`). You increment `version` when you change the chart. You change `appVersion` when the application releases a new version.

values.yaml — Configuration Defaults

```
# values.yaml – complete example
replicaCount: 1

image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "" # If empty, Chart.yaml's appVersion is used (see _helpers.tpl)
imagePullSecrets: []
nameOverride: ""
fullnameOverride: ""

serviceAccount:
  create: true
  annotations: {}
  name: ""

podAnnotations: {}
podSecurityContext: {}
securityContext: {}
```



```
service:
  type: ClusterIP
  port: 80
ingress:
  enabled: false
  className: ""
  annotations: {}
  hosts:
    - host: chart-example.local
      paths:
        - path: /
          pathType: ImplementationSpecific
  tls: []
resources:
  limits:
    cpu: 100m
    memory: 128Mi
  requests:
    cpu: 50m
    memory: 64Mi
autoscaling:
  enabled: false
  minReplicas: 1
  maxReplicas: 10
  targetCPUUtilizationPercentage: 80
nodeSelector: {}
tolerations: []
affinity: {}
# Custom application config
config:
  logLevel: info
  maxConnections: 1000
```

templates/ Directory — File by File

deployment.yaml

The main workload template. Uses Go template syntax to inject values. Every hardcoded value in raw YAML becomes a template expression.

service.yaml

Exposes the deployment. Service type (ClusterIP/NodePort/LoadBalancer) and port are driven by values.

ingress.yaml

Optional Ingress resource, wrapped in an if `.Values.ingress.enabled` guard so it is only created when enabled.

helpers.tpl — Named Templates

Files beginning with `_` are NOT rendered as Kubernetes manifests. They contain named templates (partials) that can be included in other templates. This is where shared logic lives — chart name, labels, selector labels.

```

{{/*
_helpers.tpl – standard patterns
*/}}

{{/* Expand the name of the chart. */}}
{{- define "nginx-deploy-chart.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/* Create a default fully qualified app name. */}}
{{- define "nginx-deploy-chart.fullname" -}}
{{- if .Values.fullnameOverride }}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- $name := default .Chart.Name .Values.nameOverride }}
{{- if contains $name .Release.Name }}
{{- .Release.Name | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix "-" }}
{{- end }}
{{- end }}
{{- end }}

{{/* Create chart label */}}
{{- define "nginx-deploy-chart.chart" -}}
{{- printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/* Common labels */}}
{{- define "nginx-deploy-chart.labels" -}}
helm.sh/chart: {{ include "nginx-deploy-chart.chart" . }}
{{ include "nginx-deploy-chart.selectorLabels" . }}
{{- if .Chart.AppVersion }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
{{- end }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}

{{/* Selector labels */}}
{{- define "nginx-deploy-chart.selectorLabels" -}}
app.kubernetes.io/name: {{ include "nginx-deploy-chart.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}

```

```
{{- end }}
```

NOTES.txt — Post-Install Instructions

NOTES.txt is a special template rendered after install/upgrade and displayed to the user. It is NOT applied to Kubernetes — it is just printed to stdout. It supports Go template syntax so it can show dynamic values.

```
# templates/NOTES.txt
1. Get the application URL by running these commands:
{{- if .Values.ingress.enabled }}
{{- range $host := .Values.ingress.hosts }}
  http{{ if $.Values.ingress.tls }}s{{ end }}://{{ $host.host }}{{ (index $host.paths 0).path }}
{{- end }}
{{- else if contains "NodePort" .Values.service.type }}
  export NODE_PORT=$(kubectl get --namespace {{ .Release.Namespace }} \
    -o jsonpath="{.spec.ports[0].nodePort}" services {{ include "nginx-deploy-chart.fullname" . }}
  export NODE_IP=$(kubectl get nodes --namespace {{ .Release.Namespace }} \
    -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
{{- else if contains "ClusterIP" .Values.service.type }}
  export POD_NAME=$(kubectl get pods --namespace {{ .Release.Namespace }} \
    -l "app.kubernetes.io/name={{ include "nginx-deploy-chart.name" . }}" \
    -o jsonpath="{.items[0].metadata.name}")
  echo "Visit http://127.0.0.1:8080 to use your application"
  kubectl --namespace {{ .Release.Namespace }} port-forward $POD_NAME 8080:{{ .Values.service.port }}
{{- end }}
```

tests/test-connection.yaml

```
# templates/tests/test-connection.yaml
apiVersion: v1
kind: Pod
metadata:
  name: "{{ include "nginx-deploy-chart.fullname" . }}-test-connection"
  labels:
    {{- include "nginx-deploy-chart.labels" . | nindent 4 }}
  annotations:
    "helm.sh/hook": test          # This makes it a Helm test hook
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  containers:
    - name: wget
      image: busybox
      command: ['wget']
      args: ['{{ include "nginx-deploy-chart.fullname" . }}:{{ .Values.service.port }}']
  restartPolicy: Never
```

.helmignore

Works exactly like `.gitignore`. Files and patterns listed here are excluded when packaging the chart with `helm package`.

```
# .helmignore
.DS_Store
.git/
.gitignore
.bzr/
.bzrignore
.hg/
.hgignore
.svn/
*.swp
*.bak
*.tmp
*.orig
*~
.project
.idea/
*.tmproj
.vscode/
```

Chapter Summary

- ✓ `Chart.yaml`: `version` (chart SemVer) vs `appVersion` (app version) is a critical distinction.
- ✓ `type`: library charts contain only reusable named templates, no deployable manifests.
- ✓ Files starting with `_` are not rendered — they hold named templates (partials).
- ✓ `NOTES.txt` is rendered after install and printed to `stdout` — supports Go templates.
- ✓ `.helmignore` excludes files from chart packages, like `.gitignore`.

Chapter 5 — Go Templating in Helm

Why Go Templates?

Helm uses Go's text/template package (augmented with Sprig functions). Go templates were chosen because:

- **Native Go:** Helm is written in Go — no external dependency for the template engine.
- **Powerful control flow:** if/else, range loops, with blocks — sufficient for all Kubernetes configuration patterns.
- **Sprig library:** 70+ utility functions for string manipulation, math, encoding, date formatting.
- **Pipeline syntax:** Unix-pipe style function chaining: `{{ .Values.name | upper | quote }}`

Template Syntax and Whitespace Control

The fundamental syntax uses double curly braces. Whitespace control is critical because YAML is whitespace-sensitive:

```
{{ .Values.replicaCount }}      # Outputs the value; preserves surrounding whitespace
{{- .Values.replicaCount }}     # Left-trim: strips whitespace/newlines BEFORE this action
{{ .Values.replicaCount -}}     # Right-trim: strips whitespace/newlines AFTER this action
{{- .Values.replicaCount -}}    # Both: strips whitespace on both sides

# Example: why whitespace matters in YAML
# WITHOUT trimming:
metadata:
  labels:

    app: nginx                  # Extra blank line breaks YAML in some parsers

# WITH trimming using {{- }}:
metadata:
  labels:
    {{- if .Values.labels }}
    app: nginx                  # No extra blank line
    {{- end }}
```

Built-in Objects

Helm provides several built-in objects accessible in every template:

.Release — Release Information

Field	Type	Description	Example Value
.Release.Name	string	The release name	my-nginx
.Release.Namespace	string	Namespace for the release	production

Field	Type	Description	Example Value
.Release.IsInstall	bool	True on first install	true
.Release.IsUpgrade	bool	True on upgrade or rollback	false
.Release.Revision	int	Revision number (1 on install)	3
.Release.Service	string	Always 'Helm'	Helm

.Chart — Chart Metadata

Field	Description
.Chart.Name	Chart name from Chart.yaml
.Chart.Version	Chart version from Chart.yaml
.Chart.AppVersion	App version from Chart.yaml
.Chart.Description	Chart description
.Chart.Type	'application' or 'library'

.Values — User-Supplied Values

Contains merged values from values.yaml + user overrides. Access with dot notation: .Values.image.repository, .Values.replicaCount.

.Files — Non-Template Files

```
# Access non-template files in the chart directory
# Use case: embed config files into ConfigMaps

apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "mychart.fullname" . }}-config
data:
  config.yaml: |-
    {{ .Files.Get "files/config.yaml" | indent 4 }}

# Access all files matching a glob pattern
{{- (.Files.Glob "files/*.conf").AsConfig | nindent 2 }}
```

.Capabilities — Cluster Capabilities

```
# Check Kubernetes API version for compatibility
{{- if .Capabilities.APIVersions.Has "networking.k8s.io/v1/Ingress" }}
apiVersion: networking.k8s.io/v1
{{- else }}
apiVersion: extensions/v1beta1
{{- end }}

# Check Kubernetes version
{{- if semverCompare ">=1.19-0" .Capabilities.KubeVersion.GitVersion }}
# Use newer API
{{- end }}

# Check if a CRD exists
{{- if .Capabilities.APIVersions.Has "cert-manager.io/v1/Certificate" }}
# cert-manager is installed
{{- end }}
```

.Template — Current Template Info

```
# .Template.Name — path to current template file
# e.g., "mychart/templates/deployment.yaml"
# Used for error messages:
{{ fail (printf "Error in %s: image.tag is required" .Template.Name) }}

# .Template.BasePath — templates/ directory path
```

Essential Template Functions

Function	Signature	Description	Example
default	default DEFAULT VALUE	Return VALUE if non-empty, else DEFAULT	{{ .Values.tag default "latest" }}
required	required MSG VALUE	Error with MSG if VALUE is empty	{{ required "tag required" .Values.tag }}
quote	quote STRING	Wrap in double quotes	{{ .Values.name quote }}
upper / lower	upper STRING	Case conversion	{{ .Values.env upper }}
trim	trim STRING	Remove leading/trailing whitespace	{{ .Values.name trim }}
replace	replace "old" "new" STRING	String substitution	{{ .Chart.Version replace "+" "_" }}
contains	contains "substr" STRING	Returns bool	{{ if contains "nginx" .Values.image.repository }}
toYaml	toYaml VALUE	Serialize value to YAML string	{{ .Values.labels toYaml indent 4 }}

Function	Signature	Description	Example
fromYaml	fromYaml STRING	Parse YAML string to object	<code>{{ \$obj := fromYaml .Values.rawYaml }}</code>
indent	indent N STRING	Add N spaces to every line	<code>{{ .Values.config toYaml indent 4 }}</code>
nindent	nindent N STRING	Newline then indent N spaces	<code>{{ .Values.labels toYaml nindent 4 }}</code>
include	include "name" .	Render named template as string (pipe-able)	<code>{{ include "chart.labels" . nindent 4 }}</code>
tpl	tpl TEMPLATE .	Render a string as a template at runtime	<code>{{ tpl .Values.configTemplate . }}</code>
lookup	lookup "v1" "Secret" "ns" "name"	Look up a live K8s resource	<code>{{ (lookup "v1" "Secret" "ns" "name").data }}</code>
b64enc	b64enc STRING	Base64 encode	<code>{{ .Values.password b64enc }}</code>
randAlphaNum	randAlphaNum N	Random alphanumeric string (N chars)	<code>{{ randAlphaNum 32 }}</code>
sha256sum	sha256sum STRING	SHA-256 hash	<code>{{ .Values.config toYaml sha256sum }}</code>

Pipelines

```
# Pipeline: chain functions with | operator
{{ .Values.image.tag | default .Chart.AppVersion | quote }}
# 1. Get .Values.image.tag
# 2. If empty, use .Chart.AppVersion
# 3. Wrap in quotes

{{ .Values.config | toYaml | nindent 2 }}
# 1. Get config map from values
# 2. Serialize to YAML
# 3. Add newline + 2-space indent (for embedding in another YAML)

{{ .Values.name | upper | trim | replace " " "-" }}
# 1. Get name
# 2. Uppercase it
# 3. Trim whitespace
# 4. Replace spaces with hyphens
```

Control Structures

if / else / else if


```
# Basic if
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
# ... Ingress spec
{{- end }}
# if / else
{{- if eq .Values.service.type "LoadBalancer" }}
  type: LoadBalancer
{{- else if eq .Values.service.type "NodePort" }}
  type: NodePort
{{- else }}
  type: ClusterIP
{{- end }}
# Falsy values: false, 0, nil, empty string "", empty list [], empty map {}
# All other values are truthy
# Logical operators in conditions
{{- if and .Values.ingress.enabled .Values.ingress.tls }}
  # Both enabled AND tls configured
{{- end }}
{{- if or .Values.metrics.enabled .Values.tracing.enabled }}
  # Either metrics OR tracing
{{- end }}
{{- if not .Values.ingress.enabled }}
  # Ingress NOT enabled
{{- end }}
```

range — Iteration

```
# Iterate over a list
# values.yaml: extraEnvVars:
#   - name: LOG_LEVEL
#     value: info
#   - name: PORT
#     value: "8080"
env:
  {{- range .Values.extraEnvVars }}
    - name: {{ .name }}
      value: {{ .value | quote }}
  {{- end }}
# Iterate over a map (key-value)
# values.yaml: labels:
#   team: platform
#   env: production
```

```
metadata:
  labels:
    {{- range $key, $value := .Values.labels }}
    {{ $key }}: {{ $value | quote }}
    {{- end }}

# Iterate with index
{{- range $i, $host := .Values.ingress.hosts }}
  # $i = 0,1,2... ; $host = host object
{{- end }}

# Range over a list of simple strings
{{- range .Values.tolerations }}
- key: {{ .key }}
  operator: {{ .operator }}
  effect: {{ .effect }}
{{- end }}
```

with — Scope Narrowing

```
# 'with' changes the scope of '.' to the specified value
# and only executes the block if the value is non-empty

{{- with .Values.resources }}
resources:
  limits:
    cpu: {{ .limits.cpu }}      # . is now .Values.resources
    memory: {{ .limits.memory }}
  requests:
    cpu: {{ .requests.cpu }}
    memory: {{ .requests.memory }}
{{- end }}

# $ always refers to the ROOT scope, even inside with/range
{{- with .Values.podAnnotations }}
annotations:
  {{- toYaml . | nindent 4 }}
  # Access root scope with $
  managed-by: {{ $.Release.Name }}
{{- end }}
```

Named Templates: define / include vs template

```
# DEFINING a named template in _helpers.tpl:
{{- define "mychart.labels" -}}
app.kubernetes.io/name: {{ include "mychart.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
helm.sh/chart: {{ include "mychart.chart" . }}
{{- end }}

# USING include (PREFERRED – returns string, can be piped):
metadata:
  labels:
    {{- include "mychart.labels" . | nindent 4 }}
    # nindent adds a newline then 4 spaces – correct for nested YAML

# USING template (LEGACY – outputs directly, cannot be piped):
metadata:
  labels:
    {{- template "mychart.labels" . }}
    # Cannot pipe: template "foo" . | nindent 4 ← DOES NOT WORK

# RULE: Always use 'include' over 'template' so you can pipe to indent/nindent
```

■ **EXAM TIP:** Always use include instead of template. The critical difference: include returns the template output as a string, making it pipe-able (e.g., | nindent 4). template writes directly to output and cannot be piped. In production charts, virtually all named template calls should use include.

The tpl Function — Dynamic Template Evaluation

```
# tpl renders a string value AS IF it were a template
# Use case: allow users to put template expressions in values.yaml

# values.yaml:
configAnnotation: "deployed-by-{{ .Release.Name }}-v{{ .Chart.Version }}"

# template:
annotations:
  config: {{ tpl .Values.configAnnotation . }}
# Output: config: deployed-by-myapp-v1.2.3

# DANGER: tpl evaluates arbitrary template code from values
# A malicious values file could inject: {{ exec "rm -rf /" }}
# Only use tpl when you trust the values source
```

■ **COMMON MISTAKE:** The tpl function is powerful but dangerous. Only use it when you explicitly need users to write template expressions in values. Never use it on untrusted input.

Common Templating Mistakes

- **Forgetting nindent vs indent:** Use nindent N (newline + N spaces) when the output starts at the beginning of a YAML key. Use indent N when the current line already has proper indentation.
- **Missing dash in range/with ends:** `{{- end }}` strips the newline after the end — often required to avoid blank lines in YAML.
- **Using template instead of include:** You can't pipe template output. Always use include.
- **Forgetting \$ inside range/with:** Inside a range or with block, `.` changes scope. Use `$` to access the root context.
- **YAML type coercion:** `port: 8080` creates an integer. `port: '8080'` or `port: {{ .Values.port | quote }}` creates a string. Wrong types cause API server errors.

Chapter Summary

- ✓ Helm uses Go text/template + Sprig functions — learn the whitespace control dashes.
- ✓ Built-in objects: `.Release`, `.Chart`, `.Values`, `.Files`, `.Capabilities`, `.Template`
- ✓ Always use include over template — include returns a string that can be piped.
- ✓ Use nindent for embedding multi-line values — it adds newline + indent.
- ✓ `$` always refers to the root context inside range/with blocks.
- ✓ `tpl` evaluates values as templates — powerful but dangerous with untrusted input.

Chapter 6 — Values: Configuration Management

Values Precedence Hierarchy

When multiple sources supply values, Helm merges them with higher-priority sources overriding lower-priority ones. From **lowest to highest priority**:

Priority	Source	How Applied
1 (lowest)	Chart's values.yaml	Bundled with chart; always present
2	Sub-chart's values.yaml	If using sub-charts/dependencies
3	Parent chart's values.yaml	Parent can override sub-chart defaults
4	User values file (-f custom.yaml)	Multiple -f flags applied left to right
5 (highest)	--set flag(s)	Command-line overrides; override everything

```
# Example: multiple sources
# values.yaml says: replicaCount: 1
# prod-values.yaml says: replicaCount: 3
# --set flag says: replicaCount: 5

helm install myapp ./mychart \
  -f prod-values.yaml \           # priority 4: replicaCount=3
  --set replicaCount=5           # priority 5: replicaCount=5 (wins)
# Result: replicaCount = 5
```

Flags Comparison: --set vs -f vs Variants

Flag	Use For	Syntax Example	Note
--set	Simple scalar overrides	--set image.tag=v1.2.3	Shell-expanded; avoid special chars
-f / --values	Multiple values or complex structures	-f prod-values.yaml	Recommended for complex config
--set-string	Force string type (prevent YAML type coercion)	--set-string service.port=8080	Ensures '8080' not integer 8080
--set-file	Load value from a file	--set-file config=./app.conf	Reads file content as string value
--set-json	Pass JSON value (nested objects/arrays)	--set-json 'labels={"team":"ops"}'	Useful for complex nested values

```
# --set array syntax (comma-delimited)
--set tolerations[0].key=node-role,tolerations[0].operator=Exists

# --set nested object
--set image.repository=myrepo,image.tag=v2

# --set with special characters (escape with backslash)
--set "name=weird,value"      # Wrong – comma is separator
--set "name=weird\\,value"    # Correct – escaped comma

# --set-json for complex structures
--set-json 'nodeSelector={"kubernetes.io/arch":"amd64","tier":"frontend"}'

# Multiple -f flags (left-to-right priority, later files win)
helm install myapp ./chart \
  -f base-values.yaml \
  -f env/production.yaml \
  -f secrets/prod-secrets.yaml
```

Deep Merge vs Shallow Merge

Helm performs a **deep merge** for nested objects. This means you can override only one field of a nested object without losing the other fields:

```
# values.yaml:
image:
  repository: nginx
  tag: latest
  pullPolicy: IfNotPresent

# With: --set image.tag=v1.2.3
# Result (deep merge):
image:
  repository: nginx      # preserved
  tag: v1.2.3           # overridden
  pullPolicy: IfNotPresent # preserved

# EXCEPTION: Arrays are NOT deep merged – they are replaced entirely
# values.yaml: tolerations: [{key: gpu, operator: Exists}]
# With: --set tolerations[0].key=cpu
# Result: tolerations: [{key: cpu}] ← other fields in array item are LOST
```

Global Values — Cross-Sub-Chart Communication

```
# A special 'global' key in values.yaml is accessible to ALL sub-charts
# Parent chart values.yaml:
global:
  imageRegistry: "my-registry.example.com"
  imagePullSecrets:
    - name: registry-secret
  storageClass: "fast-ssd"

# In a sub-chart template:
image: {{ .Values.global.imageRegistry }}/{{ .Values.image.repository }}:{{ .Values.image.tag }}

# Sub-charts can also set their own global values:
# bitnami/postgresql's values.yaml:
global:
  postgresql:
    auth:
      postgresPassword: "" # Can be overridden from parent chart
```

Schema Validation with values.schema.json

Helm 3.x supports JSON Schema validation of values. Place values.schema.json in the chart root alongside values.yaml. Helm validates all supplied values against this schema before rendering templates.

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["image", "service"],
  "properties": {
    "replicaCount": {
      "type": "integer",
      "minimum": 1,
      "maximum": 50,
      "description": "Number of replicas"
    },
    "image": {
      "type": "object",
      "required": ["repository", "tag"],
      "properties": {
        "repository": { "type": "string" },
        "tag": { "type": "string", "pattern": "^[a-zA-Z0-9._-]+$" },
        "pullPolicy": {
          "type": "string",
          "enum": ["Always", "IfNotPresent", "Never"]
        }
      }
    }
  }
},
```

```
"service": {
  "type": "object",
  "properties": {
    "type": {
      "type": "string",
      "enum": ["ClusterIP", "NodePort", "LoadBalancer", "ExternalName"]
    },
    "port": { "type": "integer", "minimum": 1, "maximum": 65535 }
  }
}
```

Secrets in Values — Best Practices

■ **COMMON MISTAKE:** NEVER commit secrets to values.yaml or pass them with --set in production scripts. --set values appear in shell history (~/. bash_history), process lists (ps aux), and Kubernetes audit logs. The rendered secret is also stored in the Helm release Secret in plaintext (only base64, not encrypted).

Secrets management patterns:

Pattern	Tool	How It Works
Sealed Secrets	Bitnami Sealed Secrets	Encrypt secrets with cluster's public key; safe to commit to git
SOPS + Helm Secrets	Mozilla SOPS + helm-secrets plugin	Encrypt values files; decrypt at deploy time
External Secrets Operator	External Secrets Operator	Sync secrets from AWS Secrets Manager, Vault, GCP Secret Manager into K8s Secrets
Vault Agent Injector	HashiCorp Vault	Inject secrets as files/env vars via sidecar containers
Kubernetes Secrets from CI/CD	GitHub Actions, GitLab CI	Create K8s Secret before helm install; reference by name in values

Chapter Summary

- ✓ Values precedence (low→high): values.yaml → sub-chart → parent chart → -f files → --set flags.
- ✓ Use --set for quick overrides, -f for environment-specific configuration files.
- ✓ --set-string forces string type to prevent YAML integer/boolean coercion bugs.
- ✓ Deep merge preserves unspecified nested fields; arrays are fully replaced.

- ✓ Global values (`.Values.global.*`) are visible to all sub-charts.
- ✓ `values.schema.json` validates user input before template rendering — use it.
- ✓ Never commit secrets; use Sealed Secrets, SOPS, or External Secrets Operator.

PART 3: HELM CLI — COMPLETE COMMAND REFERENCE

Chapter 7 — Repository Management

What Is a Helm Repository?

A **Helm repository** is an HTTP server that serves chart packages. It must provide an `index.yaml` file listing all available charts and their versions, plus the chart archives themselves (`.tgz` files).

```
# Add a repository
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add stable https://charts.helm.sh/stable # deprecated
helm repo add jetstack https://charts.jetstack.io # cert-manager

# List configured repositories
helm repo list
# NAME          URL
# bitnami       https://charts.bitnami.com/bitnami
# jetstack      https://charts.jetstack.io

# Update local cache (fetch latest index.yaml from all repos)
helm repo update

# Remove a repository
helm repo remove bitnami

# Update a specific repo only
helm repo update bitnami
```

Searching for Charts

Command	Searches	Requires	Use Case
helm search repo	Local repo cache only	helm repo add first	Find charts in configured repos
helm search hub	Artifact Hub (artifacthub.io)	Internet access	Find any public chart globally

```
# Search configured repos
helm search repo nginx

# NAME                                CHART VERSION  APP VERSION  DESCRIPTION
# bitnami/nginx                      15.1.0         1.25.3       NGINX Open Source is a web server...

# Search with all versions
helm search repo nginx --versions

# Search Artifact Hub
helm search hub wordpress

# Show chart details
helm show chart bitnami/nginx
helm show values bitnami/nginx      # Show default values
helm show readme bitnami/nginx      # Show README
helm show all bitnami/nginx          # Show everything
```

OCI Registries — The Future of Helm Distribution

Since Helm 3.8, **OCI (Open Container Initiative) registries** are a stable alternative to HTTP chart repositories. Charts are stored as OCI artifacts alongside container images in registries like Docker Hub, AWS ECR, GCR, and Harbor.

```
# Login to an OCI registry
helm registry login registry-1.docker.io --username myuser

# Pull a chart from OCI registry (downloads .tgz to current dir)
helm pull oci://registry-1.docker.io/bitnamicharts/nginx --version 15.1.0

# Install directly from OCI
helm install my-nginx oci://registry-1.docker.io/bitnamicharts/nginx

# Push a chart to OCI registry
helm package ./mychart
helm push mychart-1.0.0.tgz oci://my-registry.example.com/charts

# OCI vs traditional repo differences:
# - No 'helm repo add' needed – reference by full OCI URL
# - Version pinning required for oci:// references
# - Integrated with existing container image tooling
```

Setting Up a Private Repository

```
# Option 1: GitHub Pages
# 1. Package your charts
helm package ./mychart
# 2. Generate index
helm repo index . --url https://myorg.github.io/helm-charts
# 3. Push charts + index.yaml to GitHub Pages branch

# Option 2: ChartMuseum (self-hosted)
helm repo add my-repo http://chartmuseum.example.com

# Option 3: Harbor (enterprise, RBAC, OCI support)
helm repo add harbor https://harbor.example.com/chartrepo/myproject \
  --username admin --password secret

# Option 4: Nexus Repository Manager
# (treat as standard HTTP repo with basic auth)
```

Chapter Summary

- ✓ `helm repo add/list/update/remove` manages local repository configuration.
- ✓ `helm search repo` searches local cache; `helm search hub` searches Artifact Hub globally.
- ✓ OCI registries (helm 3.8+) allow storing charts in container registries — no `index.yaml` needed.
- ✓ Private repos supported via HTTP basic auth, certificate auth, or OCI registry credentials.

Chapter 8 — Installing and Managing Releases

helm install — Complete Syntax

```
# Basic install
helm install <RELEASE-NAME> <CHART> [flags]

# Examples:
helm install my-nginx bitnami/nginx
helm install my-nginx ./nginx-deploy-chart      # local chart directory
helm install my-nginx ./nginx-1.0.0.tgz        # local .tgz archive
helm install my-nginx oci://registry/charts/nginx # OCI registry

# Critical flags:
helm install my-nginx bitnami/nginx \
  --namespace production \           # target namespace
  --create-namespace \               # create ns if not exists
  --values prod-values.yaml \        # custom values file
  --set image.tag=1.25.3 \           # inline overrides
  --set-string service.port=8080 \   # force string type
  --version 15.1.0 \                # specific chart version
  --dry-run \                        # simulate – renders templates, no deploy
  --debug \                          # verbose output including rendered manifests
  --wait \                           # wait for all pods/services/etc to be Ready
  --timeout 10m \                   # wait timeout (default: 5m)
  --atomic \                         # wait + auto rollback on failure
  --generate-name \                 # generate release name automatically
  --dependency-update \             # update chart deps before install
  --render-subchart-notes \         # also show sub-chart NOTES.txt
  --disable-openapi-validation \     # skip K8s schema validation (rarely needed)
  --replace \                       # reuse release name of deleted release (dangerous)
  --description "Production deploy" # custom release description
```

Understanding --dry-run vs helm template

Aspect	--dry-run	helm template
Connects to cluster?	YES — validates manifests against live API server	NO — purely local rendering
RBAC required?	YES — needs cluster access	NO — works offline
.Capabilities populated?	YES — reflects actual cluster capabilities	NO — uses defaults (unless --api-versions flag used)
lookup() function works?	YES — can look up live K8s objects	NO — always returns empty
Use in CI without cluster?	NO	YES — ideal for CI linting

Aspect	--dry-run	helm template
Best for	Final pre-deploy validation	Template debugging, CI pipeline linting

--wait and --atomic Explained

```
# --wait: helm install doesn't return until all resources are Ready
# Specifically waits for:
# - Pods: all desired replicas are Running and Ready
# - Services: LoadBalancer gets an external IP (for type: LoadBalancer)
# - Deployments/StatefulSets/DaemonSets: desired replicas ready
# - Jobs: successfully complete

helm install myapp ./chart --wait --timeout 10m

# --atomic = --wait + automatic rollback on failure
# If any resource fails to become Ready within timeout:
# 1. Helm rolls back to previous revision
# 2. helm install returns non-zero exit code
# 3. No stuck "pending-install" state

helm install myapp ./chart --atomic --timeout 5m # Production-safe install
```

■ **EXAM TIP:** Always use `--atomic` for production installs and upgrades. Without it, a failed install leaves the release in a 'failed' state that blocks subsequent installs with the same name. `--atomic` automatically cleans up on failure.

helm list — Reading the Output

```
helm list -n production      # releases in production namespace
helm list -A                # all releases in all namespaces
helm list --all              # include failed/pending releases (not just deployed)
helm list --filter "nginx.*" # regex filter on release name
helm list --date             # sort by date
helm list --output json      # JSON output for scripting
helm list --output yaml      # YAML output
```

```
# Output columns:
NAME      NAMESPACE REVISION UPDATED           STATUS  CHART      APP VERSION
my-nginx  prod       3        2024-01-15 10:30:00 deployed nginx-15.1.0 1.25.3

# STATUS values:
# deployed      - currently installed and healthy
# failed        - last operation failed
# pending-install - install in progress or stuck
# pending-upgrade - upgrade in progress or stuck
# pending-rollback - rollback in progress or stuck
# superseded    - this revision was replaced by a newer one
# uninstalled   - release was uninstalled (only with --keep-history)
```

helm status and helm get Subcommands

```
# helm status: show current state of a release
helm status my-nginx -n production

# helm get: retrieve specific release data
helm get all my-nginx           # everything below combined
helm get values my-nginx        # user-supplied values only (not defaults)
helm get values my-nginx --all  # ALL values including defaults
helm get manifest my-nginx      # rendered K8s manifests (what was actually deployed)
helm get hooks my-nginx         # hook manifests
helm get notes my-nginx         # NOTES.txt output
```

helm uninstall

```
# Uninstall a release (removes all K8s resources created by the release)
helm uninstall my-nginx -n production

# Keep release history (release secrets remain, marked 'uninstalled')
# Allows rollback to previous revisions or reinstall with same name
helm uninstall my-nginx --keep-history -n production

# Uninstall without waiting for completion
helm uninstall my-nginx --no-hooks # skip pre/post-delete hooks
```

Chapter Summary

- ✓ helm install: use --atomic for production (= --wait + auto-rollback on failure).
- ✓ --dry-run validates against the live API server; helm template is offline/local only.
- ✓ helm list STATUS column: deployed, failed, pending-install/upgrade/rollback, superseded.
- ✓ helm get manifest shows what was actually deployed; helm get values shows configured values.

✓ --keep-history on uninstall preserves history for possible re-install.

Chapter 9 — Upgrading, Rolling Back, and History

helm upgrade — Complete Syntax

```
# Basic upgrade
helm upgrade <RELEASE-NAME> <CHART> [flags]

# Key flags:
helm upgrade my-nginx bitnami/nginx \
  --namespace production \
  --values prod-values.yaml \
  --set image.tag=1.26.0 \
  --version 15.2.0 \           # upgrade to specific chart version
  --install \                 # install if release doesn't exist (upgrade-or-install)
  --atomic \                  # wait + rollback on failure
  --cleanup-on-fail \         # delete newly created resources if upgrade fails
  --reset-values \            # discard all previous values; use chart defaults + new --set
  --reuse-values \            # keep previous values; only apply new --set overrides
  --force \                   # force replace resources (deletes + recreates)
  --recreate-pods \           # deprecated in v3, use --force for pod restart
  --wait --timeout 10m \
  --description "Upgrade to v1.26.0" \
  --history-max 10             # limit history kept for this release
```

--reset-values vs --reuse-values

Flag	Behavior	Risk	When to Use
--reset-values	Discard ALL previously supplied values; start fresh from chart defaults + this upgrade's --set/--values	LOW — deterministic; explicit values always win	Clean upgrades where you supply a complete values file
--reuse-values	Merge previous release's values into this upgrade; only add new flags on top	HIGH — previous values persist invisibly; drift accumulates over time	Quick patch upgrades where only one field changes
(neither)	Default behavior: same as --reuse-values but re-merges chart defaults for new chart version keys	MEDIUM — new default values from chart are applied	Standard upgrades with a complete -f values file

■ **COMMON MISTAKE:** Avoid --reuse-values in automated pipelines. Values accumulated from multiple upgrades become invisible to the current operator, causing configuration drift. Best practice: always supply a complete -f values.yaml and use --reset-values for predictable, reproducible upgrades.

helm history — Reading Release History

```
helm history my-nginx -n production
```

#	REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
# 1		2024-01-10 09:00:00	superseded	nginx-14.0.0	1.24.0	Install complete
# 2		2024-01-12 14:00:00	superseded	nginx-15.0.0	1.25.0	Upgrade complete
# 3		2024-01-15 10:30:00	deployed	nginx-15.1.0	1.25.3	Upgrade complete

```
# STATUS column:
```

```
# deployed      - current active revision
```

```
# superseded    - replaced by newer revision
```

```
# failed        - this revision's install/upgrade failed
```

```
# uninstalled   - uninstalled (with --keep-history)
```

helm rollback — How It Works Internally

Rollback does NOT revert the cluster state to the previous Kubernetes object state. Instead, it re-runs the **install** of the previous revision's rendered manifests. This is an important distinction:

```
# Roll back to revision 2
```

```
helm rollback my-nginx 2 -n production
```

```
# Roll back to previous revision (implicit)
```

```
helm rollback my-nginx -n production
```

```
# With flags:
```

```
helm rollback my-nginx 2 \
```

```
--wait \           # wait for pods to be Ready after rollback
```

```
--timeout 5m \
```

```
--cleanup-on-fail # clean up new resources if rollback itself fails
```

```
# After rollback: a NEW revision is created (revision 4 = manifest of revision 2)
```

```
# Revision 4: STATUS=deployed, DESCRIPTION="Rollback to 2"
```

```
# Revision 3: STATUS=superseded
```

```
# Revision 2: STATUS=superseded
```

■ **NOTE:** helm rollback creates a new revision. It does NOT delete the failed revision from history. If you rollback from revision 3 to revision 2, you get revision 4 (which is a re-deployment of revision 2's manifests). This is important because databases and stateful apps may not rollback cleanly — schema migrations don't auto-revert.

Fixing Stuck Releases

```
# Problem: release stuck in 'pending-upgrade' state
# Error: another operation (install/upgrade/rollback) is in progress

# Diagnosis:
helm history my-nginx -n production
# Shows pending-upgrade status

# Fix: rollback to the last successful revision
helm rollback my-nginx 0 # 0 = latest deployed revision

# Alternative fix: update release status directly (use with caution)
kubectl patch secret sh.helm.release.v1.my-nginx.v3 \
  -n production \
  --type merge \
  -p '{"metadata":{"labels":{"status":"deployed}}}'

# Nuclear option: delete all release secrets and reinstall
kubectl delete secrets -n production -l owner=helm,name=my-nginx
helm install my-nginx bitnami/nginx -n production
```

Chapter Summary

- ✓ `helm upgrade --atomic` is the production-safe upgrade pattern (wait + auto-rollback).
- ✓ Avoid `--reuse-values` in automation — causes invisible configuration drift.
- ✓ `helm rollback` creates a NEW revision (re-runs old manifest) — does NOT revert in-place.
- ✓ Rollback number 0 rolls back to the last deployed (successful) revision.
- ✓ Stuck pending states: use `helm rollback 0` to recover.

Chapter 10 — Chart Dependencies

Why Sub-Charts Exist

Complex applications depend on other applications — a web app needs a database, a monitoring stack needs Prometheus and Grafana. Rather than deploying these separately and hoping they're installed before your app, Helm's dependency system lets you declare them as part of your chart and install everything as an atomic unit.

```
# Chart.yaml — declare dependencies
dependencies:
  - name: postgresql
    version: "12.x.x"          # SemVer constraint: >=12.0.0, <13.0.0
    repository: "https://charts.bitnami.com/bitnami"
    condition: postgresql.enabled # Only install if .Values.postgresql.enabled == true
    tags:
      - database

  - name: redis
    version: "17.x.x"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
    alias: cache                # Use 'cache' as the sub-chart name in values

# Download dependencies to charts/ directory
helm dependency update          # Downloads charts, generates Chart.lock

# Rebuild from existing Chart.lock (no network, uses locked versions)
helm dependency build

# List resolved dependencies
helm dependency list

# charts/ directory after dependency update:
charts/
■■■ postgresql-12.5.0.tgz      # downloaded sub-chart
■■■ redis-17.0.0.tgz
```

Chart.yaml vs Chart.lock

File	Purpose	Analogy	Commit to Git?
Chart.yaml (dependencies)	Declares dependency constraints (version ranges)	package.json (npm)	YES always
Chart.lock	Records exact resolved versions (checksums)	package-lock.json (npm)	YES — ensures reproducibility

File	Purpose	Analogy	Commit to Git?
charts/ directory	Downloaded dependency archives	node_modules/	Optional — can regenerate with dep build

Passing Values to Sub-Charts

```
# Parent chart's values.yaml
postgresql:
  enabled: true
  auth:
    postgresPassword: "supersecret"
    database: "myapp"
  primary:
    persistence:
      enabled: true
      size: 10Gi

cache:                # using alias 'cache' for redis
  enabled: true
  auth:
    password: "redissecret"

# In templates, sub-chart values are accessible as .Values.postgresql.*
# Sub-chart templates access them as .Values.auth.postgresPassword (their own scope)
```

Conditions and Tags

```
# Condition: enable/disable based on a single value
dependencies:
  - name: postgresql
    condition: postgresql.enabled    # .Values.postgresql.enabled

# Tags: enable/disable groups of charts
dependencies:
  - name: prometheus
    tags: [monitoring]
  - name: grafana
    tags: [monitoring]

# values.yaml to disable monitoring group:
tags:
  monitoring: false
```

Chapter Summary

✓ Chart.yaml declares dependencies; Chart.lock pins exact versions — both should be committed.

- ✓ helm dependency update downloads sub-charts; helm dependency build uses Chart.lock (offline).
- ✓ Pass values to sub-charts using the sub-chart name as a key in parent values.yaml.
- ✓ conditions enable/disable individual sub-charts; tags enable/disable groups.
- ✓ global values are accessible to all sub-charts via .Values.global.*

Chapter 11 — Testing, Linting, and Validation

```
# Lint: static analysis of chart structure and template syntax
helm lint ./mychart # basic lint
helm lint ./mychart --strict # treat warnings as errors
helm lint ./mychart -f prod-values.yaml # lint with specific values

# helm lint checks:
# - Chart.yaml required fields present and valid
# - Template syntax errors
# - NOTES.txt present
# - Values referenced in templates exist in values.yaml
# - apiVersion compatibility

# helm template: render templates locally (no cluster needed)
helm template my-release ./mychart # render all templates
helm template my-release ./mychart --debug # verbose with source info
helm template my-release ./mychart -s templates/deployment.yaml # single template
helm template my-release ./mychart > rendered.yaml # save output

# Pipe to kubeval for schema validation
helm template my-release ./mychart | kubeval
helm template my-release ./mychart | kubeconform -kubernetes-version 1.28

# Helm test: run test pods in the cluster
helm test my-release -n production # runs pods annotated with helm.sh/hook: test
helm test my-release --logs # print test pod logs after run
```

Recommended CI/CD Validation Pipeline Order

Step	Command	Purpose
1. Lint	helm lint ./chart --strict	Syntax and structure validation
2. Template	helm template release ./chart -f values.yaml > manifest.yaml	Render templates
3. Schema validate	kubeconform < manifest.yaml	Validate against K8s API schemas
4. Policy check	confest test manifest.yaml (OPA)	Enforce org policies
5. Diff (upgrade only)	helm diff upgrade release ./chart	Preview what changes
6. Deploy	helm upgrade --install release ./chart --atomic	Deploy to cluster
7. Smoke test	helm test release	Run post-deploy test pods

Chapter Summary

- ✓ `helm lint` for syntax/structure; `helm template` for local rendering; `helm test` for integration tests.
- ✓ `--dry-run` hits the API server; `helm template` is purely local — use `template` for offline CI.
- ✓ Pipe `helm template` to `kubeconform` for full Kubernetes schema validation without a cluster.
- ✓ `helm test` runs pods annotated with `helm.sh/hook: test` — runs inside the cluster.

Chapter 12 — Packaging and Distribution

```
# Package a chart into a .tgz archive
helm package ./nginx-deploy-chart
# Creates: nginx-deploy-chart-1.0.0.tgz (version from Chart.yaml)
# Package with specific output directory
helm package ./nginx-deploy-chart --destination ./releases/
# Package with signing (requires GPG key setup)
helm package ./nginx-deploy-chart --sign \
  --key "My GPG Key" \
  --keyring ~/.gnupg/pubring.gpg
# Creates: nginx-deploy-chart-1.0.0.tgz
#           nginx-deploy-chart-1.0.0.tgz.prov (provenance file)
# Verify a signed chart
helm verify nginx-deploy-chart-1.0.0.tgz
# Install a verified chart only
helm install my-nginx nginx-deploy-chart-1.0.0.tgz \
  --verify --keyring ~/.gnupg/pubring.gpg
# Generate/update repository index
helm repo index ./releases/ --url https://charts.example.com
# Push to OCI registry
helm push nginx-deploy-chart-1.0.0.tgz oci://registry.example.com/charts
```

Chapter Summary

- ✓ helm package creates a .tgz from a chart directory — version comes from Chart.yaml.
- ✓ GPG signing creates a .prov provenance file for supply chain verification.
- ✓ helm repo index generates the index.yaml needed to serve a chart repository.
- ✓ OCI push/pull works like container images — helm push/helm pull with oci:// URLs.

PART 4: ADVANCED HELM

Chapter 13 — Helm Hooks

What Are Hooks?

Hooks are Kubernetes resources (typically Jobs or Pods) that Helm executes at specific points in the release lifecycle. They allow you to perform actions that must happen before or after the main chart resources are deployed.

Real-world hook use cases:

- **pre-install:** Run database schema setup before the application starts
- **post-install:** Send a Slack notification after successful deploy
- **pre-upgrade:** Run database migration job before upgrading the application
- **post-upgrade:** Run smoke tests after upgrade
- **pre-delete:** Backup data before uninstall
- **post-delete:** Clean up external resources (DNS records, certificates)
- **test:** Run connectivity/smoke tests after install

All Hook Types

Hook Type	When It Runs	Use Case
pre-install	Before any chart resources are created	Database init, cert generation
post-install	After all chart resources are created and Ready	Notification, smoke test
pre-upgrade	Before upgrade resources are applied	Database migration, backup
post-upgrade	After upgrade is complete and resources Ready	Cache warm-up, smoke test
pre-rollback	Before rollback begins	Backup current state
post-rollback	After rollback is complete	Notification
pre-delete	Before uninstall begins	Data backup, graceful drain
post-delete	After all release resources deleted	Clean up external resources
test	Only when 'helm test' is explicitly run	Integration/smoke tests

Hook Annotations

```
# templates/db-migration.yaml — pre-upgrade hook Job
apiVersion: batch/v1
```

```
kind: Job
metadata:
  name: {{ include "mychart.fullname" . }}-db-migrate
  labels:
    {{- include "mychart.labels" . | nindent 4 }}
  annotations:
    "helm.sh/hook": pre-upgrade           # Hook type(s) – comma-separate for multiple
    "helm.sh/hook-weight": "-5"           # Execution order (-lower = earlier)
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  template:
    metadata:
      name: db-migrate
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          command: ["python", "manage.py", "migrate"]
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: {{ include "mychart.fullname" . }}-db-secret
                  key: url
```

Hook Weights — Execution Ordering

When multiple hooks of the same type exist, `helm.sh/hook-weight` controls execution order. Hooks with lower (more negative) weights run first. Default weight is 0.

```
# Execution order: -10 → -5 → 0 → 5 → 10

# Hook 1: runs first (weight -10)
annotations:
  "helm.sh/hook": pre-upgrade
  "helm.sh/hook-weight": "-10" # must be a string

# Hook 2: runs second (weight -5)
annotations:
  "helm.sh/hook": pre-upgrade
  "helm.sh/hook-weight": "-5"

# Hook 3: runs last (weight 5)
annotations:
  "helm.sh/hook": pre-upgrade
  "helm.sh/hook-weight": "5"

# Hooks with same weight run in alphabetical order by resource name
```

Hook Delete Policies

Policy	When Resource Is Deleted	Use Case
before-hook-creation (default)	Before a new hook of the same name is created	Prevents accumulation of old job pods
hook-succeeded	After the hook completes successfully	Clean cleanup on success; keep for debugging on failure
hook-failed	After the hook fails	Automatically clean up failed hooks

```
# Multiple policies: comma-separated
"helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
# This means: delete before creating a new one AND delete after success
# Failed hooks are kept for debugging
```

Hook Failure Modes

If a hook **fails** (non-zero exit code, timeout), Helm marks the entire release as failed and stops processing. The release is NOT rolled back automatically unless `--atomic` was used. To proceed after a hook failure:

```
# Find and delete failed hook jobs manually
kubectl get jobs -n production -l heritage=Helm
kubectl delete job my-app-db-migrate -n production

# Then retry the upgrade
helm upgrade myapp ./chart -n production
```

Chapter Summary

- ✓ Hooks are K8s resources (Jobs/Pods) with `helm.sh/hook` annotation — run at lifecycle points.
- ✓ Hook types: pre/post install, upgrade, rollback, delete, and test.
- ✓ `hook-weight` controls execution order (lower = earlier, strings required).
- ✓ `hook-delete-policy` controls cleanup: `before-hook-creation` + `hook-succeeded` is standard.
- ✓ A failed hook marks the release as failed — use `--atomic` to auto-rollback.

Chapter 14 — Helm Secrets & Security

How Helm Stores Release State as Secrets

Every Helm release revision is stored as a Kubernetes Secret. The Secret contains the full rendered manifest, the values used, and metadata — all base64+gzip encoded (NOT encrypted). Anyone with get secrets permission can read all release data.

```
# List Helm release secrets
kubectl get secrets -n production --field-selector type=helm.sh/release.v1

# Full decode sequence:
SECRET_NAME="sh.helm.release.v1.myapp.v3"
kubectl get secret $SECRET_NAME -n production \
  -o jsonpath='{.data.release}' | \
  base64 -d | \      # Decode Kubernetes Secret base64
  base64 -d | \      # Decode Helm's internal base64 encoding
  gunzip | \         # Decompress gzip
  python3 -m json.tool # Pretty-print JSON

# The JSON contains:
# - chart: chart metadata
# - config: user-supplied values (including any --set secrets!)
# - manifest: rendered YAML manifests
# - info: deploy time, status, description
# - version: revision number
```

Security Implications

■ **COMMON MISTAKE:** If you pass `--set password=mypassword`, that password is stored in the Helm release Secret in plaintext (just base64 encoded). Anyone with `kubectl get secrets` permission in that namespace can read it. Additionally, the value appears in shell history and CI/CD logs.

RBAC for Helm


```
# Minimum RBAC for helm install/upgrade (cluster-level for CRDs):
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: helm-deployer
  namespace: production
rules:
  # Manage Helm release secrets
  - apiGroups: [""]
    resources: ["secrets"]
    verbs: ["get", "list", "create", "update", "delete"]
  # Deploy application resources
  - apiGroups: ["apps"]
    resources: ["deployments", "replicasets"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  - apiGroups: [""]
    resources: ["services", "configmaps", "serviceaccounts"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  - apiGroups: ["networking.k8s.io"]
    resources: ["ingresses"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
```

Supply Chain Security — Chart Signing

```
# 1. Sign a chart with GPG
helm package ./mychart --sign \
  --key "Safdar Ali Shah" \
  --keyring ~/.gnupg/secring.gpg
# Creates: mychart-1.0.0.tgz + mychart-1.0.0.tgz.prov

# 2. The .prov file contains:
# - SHA-256 digest of the .tgz
# - PGP signature by the signer's key
# - Chart metadata

# 3. Verify before install
helm verify mychart-1.0.0.tgz --keyring ~/.gnupg/pubring.gpg

# 4. Install with verification
helm install myapp mychart-1.0.0.tgz --verify \
  --keyring ~/.gnupg/pubring.gpg
```

Chapter Summary

- ✓ Helm release secrets contain full rendered manifests and values — only base64, not encrypted.
- ✓ Never use --set for passwords — appears in shell history, CI logs, and release secrets.

- ✓ Use Sealed Secrets, SOPS+helm-secrets, or External Secrets Operator for production secret management.
- ✓ Minimum RBAC: get/list/create/update/delete on Secrets plus resources the chart creates.
- ✓ GPG chart signing provides supply chain verification via .prov provenance files.

Chapter 15 — Helm in CI/CD Pipelines

```
# Complete production CI/CD pipeline (GitHub Actions pattern):
# Step 1: Lint
helm lint ./chart --strict --values values/base.yaml

# Step 2: Render templates
helm template $RELEASE ./chart \
  --values values/base.yaml \
  --values values/$ENV.yaml > rendered.yaml

# Step 3: Schema validate
kubeseconform -kubernetes-version $(kubectl version --short | grep Server | awk '{print $3}' | tr -d ' ')
  rendered.yaml

# Step 4: Policy check
conftest test rendered.yaml --policy ./opa-policies/

# Step 5: Diff (upgrade only, not first install)
helm diff upgrade $RELEASE ./chart \
  --values values/base.yaml \
  --values values/$ENV.yaml \
  --allow-unreleased # don't fail if release doesn't exist yet

# Step 6: Deploy
helm upgrade --install $RELEASE ./chart \
  --namespace $NAMESPACE \
  --create-namespace \
  --values values/base.yaml \
  --values values/$ENV.yaml \
  --atomic \
  --timeout 10m \
  --description "Deploy by CI: commit $COMMIT_SHA"

# Step 7: Smoke test
helm test $RELEASE --namespace $NAMESPACE --timeout 5m
```

GitOps: ArgoCD and Flux with Helm

ArgoCD and Flux do NOT run helm install directly. Instead they run helm template to render manifests and then apply them with kubectl apply. This means:

- Helm hooks do NOT run in ArgoCD (hooks are annotations on manifests; ArgoCD uses kubectl apply, not helm install)
- NOTES.txt is not displayed
- No Helm release history (no helm history command)
- ArgoCD uses Argo's own sync tracking instead of Helm's release secrets
- Flux HelmRelease CRD does run helm install/upgrade — closer to native Helm

Helmfile — Declarative Multi-Release Management

```
# helmfile.yaml — manage many releases declaratively
repositories:
  - name: bitnami
    url: https://charts.bitnami.com/bitnami
environments:
  production:
    values:
      - env/production.yaml
  staging:
    values:
      - env/staging.yaml
releases:
  - name: my-app
    namespace: production
    chart: ./charts/my-app
    values:
      - values/base.yaml
      - values/{{ .Environment.Name }}.yaml
    set:
      - name: image.tag
        value: {{ requiredEnv "IMAGE_TAG" }}
  - name: postgresql
    namespace: production
    chart: bitnami/postgresql
    version: "12.x.x"
    values:
      - values/postgresql.yaml
    needs:
      - production/my-app # dependency ordering

# Run helmfile:
helmfile apply           # install/upgrade all releases
helmfile -e production apply # specific environment
helmfile diff            # diff all releases
helmfile test            # run all helm tests
helmfile destroy         # uninstall all releases
```

Chapter Summary

- ✓ CI/CD pipeline order: lint → template → validate → diff → deploy --atomic → test.
- ✓ ArgoCD renders Helm templates with helm template; hooks do NOT run in ArgoCD.
- ✓ Flux HelmRelease CRD runs native helm upgrade — hooks work, history maintained.

✓ Helmfile manages multiple releases declaratively across environments — fills the gap.

Chapter 16 — Chart Best Practices (Official Helm Guidelines)

Best Practice	Rule	Rationale
Chart naming	Lowercase letters and hyphens only; no underscores or dots	Kubernetes resource naming compatibility
Image tags	Never use 'latest' as default in values.yaml	Unpredictable — different pulls get different versions
Resource naming	Always use {{ include 'chart.fullname' . }}	Prevents name collisions between releases in same namespace
Labels	Always include helm.sh/chart, app.kubernetes.io/name, app.kubernetes.io/instance, app.kubernetes.io/managed-by	Required for helm list, upgrade, uninstall to work correctly
Resource limits	Always define both requests AND limits	Prevents noisy-neighbor issues and cluster resource exhaustion
Health probes	Always include liveness and readiness probes	Kubernetes can't detect unhealthy apps without probes
ServiceAccount	Always create a dedicated ServiceAccount	Principle of least privilege; don't use default SA
HPA	Include HorizontalPodAutoscaler but disabled by default	Ready to enable without chart changes
PDB	Include PodDisruptionBudget for production charts	Prevents all pods being evicted during node drain
NetworkPolicy	Include as opt-in (disabled by default)	Security benefit, but requires CNI support
NOTES.txt	Always include — document access instructions	First thing users see after install
values.schema.json	Add JSON Schema for all charts	Catch misconfiguration before deploy

```
# Standard labels in every resource template:
metadata:
  labels:
    helm.sh/chart: {{ include "mychart.chart" . }}
    app.kubernetes.io/name: {{ include "mychart.name" . }}
    app.kubernetes.io/instance: {{ .Release.Name }}
    app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
    app.kubernetes.io/managed-by: {{ .Release.Service }}
# For Deployments/StatefulSets – selector labels NEVER change after first deploy:
selector:
  matchLabels:
    app.kubernetes.io/name: {{ include "mychart.name" . }}
    app.kubernetes.io/instance: {{ .Release.Name }}
```

■ **COMMON MISTAKE:** Selector labels (matchLabels) in Deployments are IMMUTABLE after creation. Never put helm.sh/chart or version labels in the selector — these change with every upgrade and would make the Deployment unupgradeable. Only put name and instance in selectors.

Chapter Summary

- ✓ Never use 'latest' image tag — pin to appVersion or a specific digest.
- ✓ Resource naming via fullname helper prevents namespace collisions.
- ✓ Selector labels are immutable — only app name and instance, never version.
- ✓ Always define resource requests AND limits; include readiness/liveness probes.
- ✓ Include HPA, PDB, NetworkPolicy — disabled by default but ready to enable.

PART 5: REAL-WORLD WORKFLOWS

Chapter 17 — The nginx Deployment Workflow

Step 1: Generate the Deployment Manifest

```
# Generate deployment YAML from a running deployment (or use --dry-run)
kubectl create deploy nginx --image=nginx:alpine --port=80 -o yaml > deploy.yaml

# Key points:
# -o yaml: output manifest YAML (does NOT deploy)
# --port=80: sets containerPort (informational, not enforcement)
# The output contains many auto-generated fields (resourceVersion, etc.)
# Strip these for use as a Helm template
# Generated deploy.yaml (key sections):
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx          # ← will become {{ include "chart.fullname" . }}
  namespace: default   # ← will become {{ .Release.Namespace }}
spec:
  replicas: 1          # ← will become {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: nginx        # ← will become selector labels from _helpers.tpl
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:alpine # ← will become {{ .Values.image.repository }}:{{ .Values.image.tag }}
          ports:
            - containerPort: 80 # ← will become {{ .Values.service.port }}
```

Step 2: Generate the Service Manifest

```
# Generate service YAML using dry-run (does NOT deploy)
kubectl expose deploy nginx --target-port=80 -o yaml --dry-run=client > deploy-expose-service.yaml

# --dry-run=client: generates YAML locally without hitting API server
# Note: kubectl expose can only expose an EXISTING deployment without --dry-run
# With --dry-run=client, it generates the Service template without the deployment existing

# Generated service YAML (simplified):
apiVersion: v1
kind: Service
metadata:
  name: nginx          # ← will become {{ include "chart.fullname" . }}
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80    # ← will become {{ .Values.service.port }}
  selector:
    app: nginx          # ← must match pod labels
    type: ClusterIP     # ← will become {{ .Values.service.type }}
```

Step 3: Create the Helm Chart

```
# Create chart scaffold
helm create nginx-deploy-chart

# Scaffold generates default deployment/service/ingress templates
# Strategy: REPLACE the scaffolded templates with your manifests,
# then parameterize them

# File cleanup – remove templates you don't need:
rm nginx-deploy-chart/templates/hpa.yaml          # unless you want autoscaling
rm nginx-deploy-chart/templates/ingress.yaml      # unless needed

# Replace deployment.yaml with your converted template
```

Step 4: Convert Raw YAML to Helm Templates

```
# nginx-deploy-chart/templates/deployment.yaml – CONVERTED
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "nginx-deploy-chart.fullname" . }}
  namespace: {{ .Release.Namespace }}
  labels:
    {{- include "nginx-deploy-chart.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
```

```

matchLabels:
  {{- include "nginx-deploy-chart.selectorLabels" . | nindent 6 }}
template:
  metadata:
    labels:
      {{- include "nginx-deploy-chart.selectorLabels" . | nindent 8 }}
    {{- with .Values.podAnnotations }}
    annotations:
      {{- toYaml . | nindent 8 }}
    {{- end }}
  spec:
    {{- with .Values.imagePullSecrets }}
    imagePullSecrets:
      {{- toYaml . | nindent 8 }}
    {{- end }}
    serviceAccountName: {{ include "nginx-deploy-chart.serviceAccountName" . }}
    containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}
        ports:
          - name: http
            containerPort: {{ .Values.service.port }}
            protocol: TCP
        livenessProbe:
          httpGet:
            path: /
            port: http
        readinessProbe:
          httpGet:
            path: /
            port: http
        {{- with .Values.resources }}
        resources:
          {{- toYaml . | nindent 12 }}
        {{- end }}

```

```

# nginx-deploy-chart/values.yaml — COMPLETE
replicaCount: 1

image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "alpine"      # specific tag; appVersion used if empty
imagePullSecrets: []
nameOverride: ""
fullnameOverride: ""

```

```
serviceAccount:
  create: true
  annotations: {}
  name: ""
podAnnotations: {}
resources:
  limits:
    cpu: 100m
    memory: 128Mi
  requests:
    cpu: 50m
    memory: 64Mi
service:
  type: ClusterIP
  port: 80
nodeSelector: {}
tolerations: []
affinity: {}
```

Step 5: Install, Upgrade, Rollback

```
# Install the release
helm install nginx-release ./nginx-deploy-chart -n default

# Verify
helm list -n default
helm status nginx-release -n default
kubectl get pods -n default -l app.kubernetes.io/name=nginx-deploy-chart

# Upgrade: change replica count
helm upgrade nginx-release ./nginx-deploy-chart \
  --set replicaCount=3 -n default

# Verify upgrade
helm history nginx-release -n default

# Rollback to previous
helm rollback nginx-release -n default

# Package and distribute
helm package ./nginx-deploy-chart
# Creates: nginx-deploy-chart-0.1.0.tgz
```

Chapter 18 — Troubleshooting Helm

Error / Symptom	Root Cause	Fix
Release stuck in pending-install	Previous install timed out or was interrupted	helm rollback 0 OR delete all release secrets
Error: another operation in progress	Concurrent helm operations; stuck pending-upgrade	helm rollback 0 to reset to last deployed
Error: release has no deployed releases	All revisions are failed/uninstalled; no deployed revision to rollback to	helm uninstall ; helm install fresh
Hook job blocking release	pre/post hook Job failed or is still running	Delete the hook Job manually; retry upgrade
nil pointer evaluating .Values.x.y	Values path doesn't exist — parent key is nil	Use default: {{ .Values.x.y default 'fallback' }} or check with 'with'
YAML indentation errors after template	Missing nindent/indent on multi-line values	Add nindent N after toYaml in pipelines
Error: cannot re-use a name still in use	Trying to install with name of existing release	helm upgrade --install or choose different name
Helm list shows no releases	Looking in wrong namespace	helm list -A for all namespaces
Resources missing after upgrade	Resource deleted from templates — Helm 3 doesn't delete by default	Use helm diff before upgrade to detect removed resources

```
# Debugging workflow:
# 1. Render templates locally
helm template my-release ./chart --debug 2>&1 | head -100

# 2. Dry run against cluster
helm install my-release ./chart --dry-run --debug 2>&1

# 3. Check what was actually deployed
helm get manifest my-release -n production

# 4. Compare desired vs actual
helm diff upgrade my-release ./chart -f values.yaml

# 5. Check pod events
kubectl describe pod <pod-name> -n production

# 6. Check release history
helm history my-release -n production
```

PART 6: EXAM PREPARATION

Chapter 19 — CKA / CKAD / KSNA Exam Questions & Answers

All 95 exam questions answered in full. Questions are grouped by category.

SECTION A: Conceptual / Why Questions (Q1–Q15)

Q1. Why was Helm created? What specific problem does it solve that kubectl apply -f does not?

kubectl apply -f deploys resources but provides NO lifecycle management: no versioning (you can't ask 'what version is running?'), no rollback (to roll back you must manually track and re-apply old files), no templating (separate files for dev/prod), no release naming (no concept of 'an instance'), and no dependency management. Helm adds all of these: it packages manifests into Charts, creates named Releases with revision history, enables templated parameterization for multi-environment deployment, and manages dependencies between charts. In short: kubectl apply is like copying files; Helm is like a proper package manager.

Q2. Explain the difference between a Chart, a Release, and a Revision.

A CHART is the package — the directory/archive containing templates, values, and metadata. It is the blueprint. One chart can be installed many times.

A RELEASE is a named instance of a chart deployed to a specific namespace. Installing bitnami/mysql twice creates two releases: 'prod-db' and 'staging-db'. Each release tracks its own state independently.

A REVISION is a numbered snapshot of a release at a point in time. Every install creates revision 1. Every upgrade or rollback creates a new revision. Revision history enables rollback.

Q3. Why was Tiller removed in Helm v3?

Tiller ran as a server-side pod inside the cluster, typically with cluster-admin privileges. This created severe security problems: (1) Anyone who could reach Tiller could perform any Kubernetes action, bypassing RBAC entirely. (2) Helm's own permission model was separate from Kubernetes RBAC, creating two access control systems. (3) Tiller was a single point of failure and a high-value attack target. Helm v3 removed Tiller completely — the helm CLI now communicates directly with the K8s API server using the user's own kubeconfig credentials, so Kubernetes RBAC applies natively.

Q4. How is Helm different from Kustomize? When would you choose one over the other?

Helm uses Go templates with a full programming language (conditionals, loops, functions, variables). Kustomize uses YAML patching with no templating — you write overlays that patch base YAML. Choose Helm when: you need parameterization (different replicas per env), you need dependency management (app + database together), you want to distribute reusable packages, or you need

release lifecycle management (upgrade/rollback). Choose Kustomize when: you want to stay in pure YAML, you need simple overlays without logic, your team is uncomfortable with Go template syntax, or you want to compose resources without a packaging step. They can be combined: Kustomize can invoke helm template as a generator.

Q5. Why does Helm store release state as Kubernetes Secrets by default?

Secrets are stored in etcd. If etcd is encrypted at rest (common in production), Helm release data is encrypted automatically. Secrets also have a type field (helm.sh/release.v1) that allows easy filtering. Alternatives: ConfigMaps are human-readable but not encrypted. Memory storage is lost on process exit (testing only). SQL backend works for large deployments with many releases where etcd storage limits become a concern. The tradeoff: anyone with 'get secrets' permission in a namespace can read all Helm release metadata including values and rendered manifests.

Q6. What is the difference between appVersion and version in Chart.yaml?

version is the CHART package version following SemVer 2. Increment it when you change templates, values structure, or chart logic. This is what helm upgrade uses to track changes. appVersion is INFORMATIONAL ONLY — it documents what application version the chart was designed for (e.g., nginx:1.25.0). Helm does not use appVersion for any logic. The actual container image tag is set in values.yaml. Example: you might update the chart to add a PodDisruptionBudget (bump version: 1.0.1) while keeping appVersion: 1.25.0 unchanged because the nginx version didn't change.

Q7. Why should you never use 'latest' as an image tag in a Helm chart's default values.yaml?

Using 'latest' creates several problems: (1) Non-determinism: two helm install commands at different times will pull different image versions, making deployments unpredictable. (2) No rollback: helm rollback re-deploys the same manifest but 'latest' now points to a different image than when that revision was created. (3) CI/CD pipeline unreliability: you can't reproduce a deployment state. (4) Kubernetes imagePullPolicy IfNotPresent: if the node already has 'latest' cached (even an older version), the new image won't be pulled. Best practice: use a specific tag or SHA digest, e.g., tag: '1.25.3' or tag: 'sha256:abc123...'

Q8. What is the difference between helm template and helm install --dry-run?

helm template renders Go templates to YAML locally without connecting to the cluster. It never hits the Kubernetes API. .Capabilities is not populated from the real cluster (uses static defaults). lookup() returns empty. Best for offline CI linting.

helm install --dry-run connects to the Kubernetes API server. It validates rendered manifests against the live OpenAPI schema. .Capabilities reflects real cluster capabilities. lookup() works. But it does NOT create any resources. Best for final pre-deploy validation when cluster access is available.

Q9. Why does Helm use Go templates instead of a simpler format?

Go templates (with the Sprig library) provide: (1) Full control flow: if/else, range loops, with scope — necessary for conditional Ingress, optional sidecars, iterating over lists. (2) Functions: 70+ Sprig functions for strings, encoding, math, dates. (3) Pipelines: Unix-pipe chaining for value transformations. (4) Named templates: reusable partials via define/include. (5) No external dependency: Helm is written in Go so the template engine is built-in. Simpler formats like Mustache lack conditionals and functions; Jinja2 would require Python; Jsonnet has a steeper learning curve for K8s users.

Q10. What problem do Helm hooks solve? Give three real-world use cases.

Helm hooks solve the problem of needing to perform actions at specific points in the release lifecycle — actions that must complete before or after the main resources are deployed.

Real-world use cases: (1) DATABASE MIGRATION (pre-upgrade hook): Run a Django 'manage.py migrate' Job before deploying the new application version. Ensures the database schema is updated before pods start. (2) SMOKE TEST (post-install hook): After installing, run a test pod that curls the service endpoint and verifies a 200 response. (3) DATA BACKUP (pre-delete hook): Before uninstalling a stateful application, run a Job that dumps the database to S3. Ensures data safety before destroying resources.

Q11. Why is --reuse-values during upgrade potentially dangerous?

--reuse-values takes all values from the previous release revision and merges them with any new --set flags. Over multiple upgrades, values from months or years ago persist invisibly. Problems: (1) Configuration drift: an operator sets --set debug=true in an emergency fix; all future upgrades reuse debug=true without anyone knowing. (2) Invisible values: helm get values my-release shows user-supplied values including reused ones, but it's easy to miss. (3) New chart defaults ignored: if a new chart version adds a new values.yaml key with a sensible default, --reuse-values may cause the old merged values to override it unexpectedly. Best practice: always supply a complete -f values.yaml and use --reset-values.

Q12. What is the purpose of _helpers.tpl and why is it a best practice?

_helpers.tpl stores named template definitions (using {{- define ... -}}) that are reused across multiple templates in the chart. Best practice reasons: (1) DRY principle: define chart name, full name, labels, selector labels once; use everywhere. (2) Consistency: all resources get identical labels from one source of truth. (3) The underscore prefix means it is NOT rendered as a Kubernetes manifest. (4) Standard Helm charts (helm create scaffold) always include _helpers.tpl with standard named templates. If you define labels directly in each template, any change requires editing every file.

Q13. Explain the values precedence hierarchy in Helm from lowest to highest priority.

- 1 (LOWEST): Chart's values.yaml — bundled defaults.
- 2: Parent chart values.yaml for sub-chart keys.
- 3: User -f values file(s) — multiple -f flags applied left to right.
- 4 (HIGHEST): --set flags — always win.

Example: If `values.yaml` has `replicaCount: 1`, and `-f prod.yaml` has `replicaCount: 3`, and `--set replicaCount=5`, then result is 5. This hierarchy means you can provide sensible defaults in `values.yaml`, environment-specific files via `-f`, and emergency overrides via `--set`.

Q14. Why is helm rollback not always safe?

helm rollback re-deploys the previous revision's rendered manifests. Problems: (1) Database schema migrations: if upgrade ran a database migration that added a column, rollback won't revert the schema. The old application code may be incompatible with the new schema. (2) Stateful data: `PersistentVolumeClaims` and data in databases are NOT reverted. Only K8s object definitions are re-applied. (3) External resources: if the chart creates DNS records, certificates, or external cloud resources, rollback may not clean those up. (4) Immutable fields: some K8s fields (selector labels, `StorageClass` on PVCs) cannot be changed — rollback may fail with 'field is immutable'. Rollback is safe for stateless applications; approach stateful rollbacks with extreme caution.

Q15. What is the difference between helm dependency update and helm dependency build?

helm dependency update: Reads `Chart.yaml` dependencies, resolves the latest matching versions, downloads charts from repositories into the `charts/` directory, and regenerates `Chart.lock` with exact pinned versions. Requires internet/repo access.

helm dependency build: Reads `Chart.lock` (exact pinned versions), downloads exactly those versions into `charts/`. Does NOT re-resolve or update versions. Works offline if charts are already cached. Use in CI/CD where you want exactly the same dependencies as `Chart.lock` specifies — not the latest matching version.

SECTION B: Architecture / Internals Questions (Q16–Q25)

Q16. How does Helm v3 communicate with the Kubernetes API server?

Helm v3 uses the same client libraries as `kubectl` (`client-go`). Request flow:

1. helm CLI reads `$KUBECONFIG` or `~/.kube/config` to get cluster endpoint, credentials, and context.
2. helm renders Go templates to produce plain Kubernetes YAML manifests.
3. helm sends REST API calls directly to the Kubernetes API server (e.g., `POST /apis/apps/v1/namespaces/default/deployments`).
4. Helm uses the user's credentials (certificate, bearer token, or OIDC) — same as `kubectl`.
5. Kubernetes RBAC evaluates the request against the user's permissions.
6. helm stores the release record as a `Secret` via the `Secrets API`.

No proxy, no intermediate component — pure direct API server communication.

Q17. Where does Helm store release state? How do you read a Helm release secret with kubectl?

Helm stores release state in Kubernetes Secrets (default) in the release's namespace. Each revision is a separate Secret.

Full decode sequence:

```
kubectl get secrets -n --field-selector type=helm.sh/release.v1
```

```
kubectl get secret sh.helm.release.v1..v -n -o jsonpath='{.data.release}' | base64 -d | base64 -d |  
gunzip | python3 -m json.tool
```

The double base64 decode is because Kubernetes Secret data is base64-encoded, and Helm's own JSON payload is also base64-encoded before storage.

Q18. What is the structure of a Helm release secret?

The decoded JSON contains: name (release name), namespace, info (status, first_deployed, last_deployed, description), chart (Chart.yaml content, templates, schema), config (user-supplied values — THIS INCLUDES SECRETS PASSED VIA --set), manifest (full rendered YAML of all K8s resources), hooks (hook manifest list), version (revision number). The presence of config (user values) in plain text is why passing secrets via --set is dangerous.

Q19. How does Helm handle RBAC?

Helm uses the current user's kubeconfig credentials for all API calls. Required permissions depend on what the chart creates. At minimum: get/list/create/update/delete on Secrets (for release storage) + permissions for all resource types the chart manages (Deployments, Services, ConfigMaps, etc.). For cluster-scoped resources (ClusterRole, CRD), ClusterRole permissions are needed. Best practice: create a dedicated ServiceAccount with a Role/RoleBinding scoped to the deployment namespace.

Q20. Explain how helm search hub works differently from helm search repo.

helm search repo queries the LOCAL CACHE of configured repositories. You must first run helm repo add and helm repo update to populate the cache. Only finds charts from repos you've explicitly added.

helm search hub queries ARTIFACT HUB (artifacthub.io) over the internet. Artifact Hub is a CNCF-hosted index of Helm charts, operators, and plugins from hundreds of public repositories. No local repo configuration needed. Returns results from any public chart in the ecosystem.

Q21. What is an OCI registry in the context of Helm?

OCI (Open Container Initiative) registries store arbitrary artifacts, not just container images. Helm 3.8+ allows storing charts as OCI artifacts in registries like Docker Hub, AWS ECR, GCR, and Harbor. Key differences from HTTP chart repos: No index.yaml — charts are addressed by registry URL + name + tag. Use helm pull oci://registry/chart:version instead of helm repo add. Authentication uses helm registry login (same as docker login). Version is required (no 'latest' concept). Benefit: single unified registry for both images and charts.

Q22. Explain the Chart.lock file. When is it generated and when should you commit it?

Chart.lock is generated by helm dependency update. It records exact resolved versions of all dependencies (name, version, repository URL, digest/checksum). Similar to package-lock.json in npm. Generated when: you run helm dependency update. Should commit to git: YES. By committing Chart.lock, helm dependency build can reproduce the exact same dependency versions in any environment without re-resolving. Do NOT commit the charts/ directory itself (regenerate with helm dependency build).

Q23. How does Helm resolve template rendering order within a chart?

Helm renders templates in alphabetical order by filename within the templates/ directory. However, Kubernetes itself determines the apply order. Helm does NOT guarantee that resource A is Running before resource B is created — that's what --wait and hooks are for. Files starting with _ (like _helpers.tpl) are never rendered as manifests. NOTES.txt is rendered last and printed to stdout only, never applied to K8s.

Q24. What happens internally when you run helm upgrade --atomic?

1. helm loads chart templates and merges values.
2. helm renders templates to K8s manifests.
3. helm sends manifests to the API server (apply/patch).
4. --wait: helm polls K8s until all Deployment/StatefulSet/DaemonSet replicas are Ready, Services have endpoints, Jobs complete. Timeout is controlled by --timeout.
5. If timeout exceeded OR any resource enters a failed state: helm automatically runs rollback to the previous deployed revision.
6. After rollback: helm upgrade returns a non-zero exit code.
7. If everything becomes Ready within timeout: release is marked deployed, revision incremented.

Q25. How does helm rollback work at the Kubernetes object level?

helm rollback does NOT use any K8s rollback API. It works by: 1. Looking up the release Secret for the target revision. 2. Extracting the rendered manifest from that revision's Secret. 3. Running essentially a new helm upgrade using that old manifest. 4. Creating a new revision with the old manifest content and status 'deployed'. Result: new revision number, but same manifests as the target revision. This means ConfigMaps, Secrets, Deployments, Services all get re-applied as they were in the target revision. BUT: PersistentVolumeClaims and their data are NOT reverted. External state (databases, cloud resources) is NOT reverted.

SECTION C: Templating Questions (Q26–Q40)**Q26. What is the difference between include and template in Helm?**

Both render named templates defined with {{- define ... -}}.

template: outputs the rendered text DIRECTLY into the output stream. Cannot be piped. Example: {{-

template 'mychart.labels' . }} — output goes straight to the YAML; you cannot add | nindent after it.

include: returns the rendered text as a STRING that can be piped to other functions. Example: {{- include 'mychart.labels' . | nindent 4 }} — can add indentation, trimming, etc.

ALWAYS prefer include. The inability to pipe template is a fundamental limitation that causes indentation bugs.

Q27. Explain the difference between {{ }}, {{- }}, and {{- -}} in Go templates.

{{ }} — action delimiter with NO whitespace trimming. All whitespace (spaces, newlines) surrounding this block in the template file is preserved in the output.

{{- }} — LEFT trim: removes all whitespace (including newlines) BEFORE this action in the output.

{{ -}} — RIGHT trim: removes all whitespace AFTER this action.

{{- -}} — BOTH sides: removes whitespace on both sides.

This matters in YAML because extra blank lines and leading spaces affect YAML parsing. Example: {{- if .Values.enabled }} removes the newline before the if, preventing blank lines in output.

Q28. What does nindent do? Why is it used instead of indent?

indent N: adds N spaces of indentation to EVERY line of the input string. Does NOT add a leading newline.

nindent N: adds a NEWLINE first, THEN N spaces of indentation to every line. Equivalent to \n + indent N.

nindent is used in most Helm templates because when embedding a multi-line value inside YAML, you typically need the output to start on a new line relative to the parent key. Example:

```
resources:
{{- toYaml .Values.resources | nindent 2 }}
```

Without the leading newline (using indent), the first line of resources would appear on the same line as 'resources:' which is invalid YAML.

Q29. Write a template snippet that makes a value required and fails with a custom message.

In values.yaml, leave the field empty or commented out. In the template:

```
{{ $image := required "ERROR: .Values.image.tag is required. Set it with --set image.tag="
.Values.image.tag }}
```

```
image: "{{ .Values.image.repository }}:{{ $image }}"
```

```
Or inline: image: "{{ .Values.image.repository }}:{{ required \"image.tag is required\" .Values.image.tag
  }}"
```

The required function returns the value if non-empty; otherwise it causes template rendering to fail with the provided error message. This is the standard pattern for enforcing mandatory values.

Q30. Explain the .Release object. List and explain all its fields.

- .Release.Name: the release name given to helm install (e.g., 'my-nginx'). Used in resource names to ensure uniqueness.
- .Release.Namespace: the namespace the release is installed into.
- .Release.IsInstall: boolean true only during the initial helm install (not upgrades).
- .Release.IsUpgrade: boolean true during helm upgrade or helm rollback.
- .Release.Revision: integer revision number. Starts at 1 on install, increments on every upgrade/rollback.
- .Release.Service: always the string 'Helm'. Used in the app.kubernetes.io/managed-by label.

Q31. Explain the .Capabilities object and give a use case.

- .Capabilities.KubeVersion.GitVersion: full K8s version string like 'v1.28.3'.
- .Capabilities.KubeVersion.Major: major version number.
- .Capabilities.KubeVersion.Minor: minor version number.
- .Capabilities.APIVersions: object with .Has(group/version/kind) method.

Use case — choosing the right Ingress API version:

```
{{- if .Capabilities.APIVersions.Has "networking.k8s.io/v1/Ingress" }}
apiVersion: networking.k8s.io/v1
{{- else }}
apiVersion: extensions/v1beta1
{{- end }}
```

Use case — CRD availability check:

```
{{- if .Capabilities.APIVersions.Has "cert-manager.io/v1/Certificate" }}
# cert-manager is installed, create Certificate resource
{{- end }}
```

Q32. How do you reference a parent chart's values from a sub-chart?

Sub-charts can only access their own values scope. To pass values from parent to sub-chart, use the sub-chart name as a top-level key in the parent's values.yaml:

```
parent/values.yaml:
```

```
postgresql:
auth:
password: 'secret'
```

Then in the postgresql sub-chart template, `.Values.auth.password` refers to the value set by the parent.

Global values: use the 'global' key in parent values.yaml. Values under 'global' are accessible in ALL sub-charts as `.Values.global.*`:

```
global:
imageRegistry: 'my-registry.io'
```

In sub-chart: `{{ .Values.global.imageRegistry }}/{{ .Values.image.repository }}`

Q33. What is the tpl function? Give a use case and explain why it must be used carefully.

tpl renders a string as a Go template at runtime, using the current context. Use case: allow users to put template expressions in values.yaml:

```
values.yaml: serviceAnnotation: 'deployed-by={{ .Release.Name }}'
template: annotation: {{ tpl .Values.serviceAnnotation . }}
```

Danger: tpl evaluates arbitrary template code. If an attacker controls the values (e.g., via a malicious values file), they could inject harmful template expressions. Also, tpl errors surface at runtime, not at lint time. Only use tpl when explicitly needed for dynamic value interpolation and you trust the values source.

Q34. How do you iterate over a list in values.yaml using range? Show environment variable example.

```
values.yaml:
extraEnvVars:
- name: LOG_LEVEL
value: info
- name: DB_HOST
value: postgres-svc

template (deployment.yaml):
env:
{{- range .Values.extraEnvVars }}
- name: {{ .name }}
value: {{ .value | quote }}
{{- end }}
```

Inside range, `.` refers to the current list element. Access fields with `.name`, `.value`, etc.

Q35. How do you iterate over a map/dictionary in `values.yaml` using range?

```
values.yaml:
podLabels:
team: platform
env: production
tier: backend

template:
metadata:
labels:
{{- range $key, $value := .Values.podLabels }}
{{ $key }}: {{ $value | quote }}
{{- end }}
```

The range `$key, $value :=` syntax unpacks each map entry into two variables. Use `$` prefix if you need to access root context inside the range block.

Q36. Write a template for a ConfigMap that reads from `.Values.config` as a map.

```
values.yaml:
config:
database_host: postgres.example.com
max_connections: '100'
log_level: info

templates/configmap.yaml:
apiVersion: v1
kind: ConfigMap
metadata:
name: {{ include 'mychart.fullname' . }}-config
data:
{{- range $key, $value := .Values.config }}
{{ $key }}: {{ $value | quote }}
{{- end }}
```

Alternatively, use `toYaml` for the entire map:

```
data:
{{- toYaml .Values.config | nindent 2 }}
```


Q37. How do you conditionally include a block in a template? Give an example with optional Ingress.

Use the if action to guard the entire resource file:

```
templates/ingress.yaml:
{{- if .Values.ingress.enabled -}}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include 'mychart.fullname' . }}
{{- with .Values.ingress.annotations }}
annotations:
  {{- toYaml . | nindent 4 }}
{{- end }}
spec:
  rules:
    {{- range .Values.ingress.hosts }}
    - host: {{ .host | quote }}
      http:
        paths:
          {{- range .paths }}
          - path: {{ .path }}
            pathType: {{ .pathType }}
          {{- end }}
        backend:
          service:
            name: {{ include 'mychart.fullname' $ }}
            port:
              number: {{ $.Values.service.port }}
          {{- end }}
    {{- end }}
  {{- end }}
```

Q38. What is values.schema.json? How does it improve chart quality?

values.schema.json is a JSON Schema file placed in the chart root. Before template rendering, Helm validates all supplied values (merged from all sources) against this schema. Benefits: (1) Early failure: misconfiguration is caught before any resources are deployed. (2) Type enforcement: ensures 'port' is an integer, 'pullPolicy' is one of [Always, IfNotPresent, Never]. (3) Required field enforcement: mark fields as required. (4) Documentation: schema serves as machine-readable documentation of all valid values. (5) IDE support: editors can provide autocomplete for chart values.

Q39. Explain the lookup function. What are its limitations?

lookup queries a LIVE Kubernetes resource from within a template. Syntax: lookup 'apiVersion' 'Kind' 'namespace' 'name'

Example:

```
{{ $secret := lookup 'v1' 'Secret' 'production' 'my-secret' }}
{{ if $secret }}
password: {{ index $secret.data 'password' }}
{{ end }}
```

Limitations: (1) Only works when helm can connect to the cluster — returns empty during helm template (offline). (2) Makes charts cluster-dependent — the same chart behaves differently on different clusters. (3) Adds complexity and potential for race conditions. (4) Cannot look up cross-namespace resources unless the service account has permissions. Use sparingly.

Q40. How do you handle default values for nested objects in values.yaml?

Define complete nested structures in values.yaml with sensible defaults:

```
values.yaml:
resources:
limits:
cpu: 100m
memory: 128Mi
requests:
cpu: 50m
memory: 64Mi
```

In templates, use 'with' to safely access nested values:

```
{{- with .Values.resources }}
resources:
{{- toYaml . | nindent 2 }}
{{- end }}
```

For deeply nested optional values, use the 'default' function:

```
{{ .Values.database.host | default 'localhost' }}
```

For required nested values, use 'required':

```
{{ required 'database.host is required' .Values.database.host }}
```

SECTION D: Commands & CLI Questions (Q41–Q55)

Q41. What does helm list -A do? How is it different from helm list -n ?

helm list -A (or --all-namespaces) lists ALL releases across ALL namespaces in the cluster. Equivalent to kubectl get pods -A.

`helm list -n production` lists only releases in the 'production' namespace.

`helm list` (no flags) lists releases in the current namespace from kubeconfig context (usually 'default').

Q42. Explain every column in the output of `helm list`.

NAME: the release name given at install time.

NAMESPACE: Kubernetes namespace the release is deployed in.

REVISION: the current active revision number.

UPDATED: timestamp of the last operation.

STATUS: `deployed` / `failed` / `pending-install` / `pending-upgrade` / `pending-rollback` / `superseded` / `uninstalled`.

CHART: chart name and chart version (e.g., `nginx-15.1.0`).

APP VERSION: the `appVersion` field from `Chart.yaml` (informational only).

Q43. What does `helm status` show? How is it different from `helm get all`?

`helm status` shows: release name, last deployed time, namespace, status (`deployed/failed`), number of resources, and `NOTES.txt` output. It is a summary view.

`helm get all` shows: full values (`helm get values --all` output), the rendered manifest (`helm get manifest`), hook manifests, and `NOTES.txt`. It is the complete technical detail of the release. Use `helm status` for a quick health check; use `helm get all` for debugging.

Q44. What is the difference between `helm get values` and `helm get values --all`?

`helm get values` : shows ONLY the user-supplied values (values explicitly passed via `-f` or `--set` flags). Does NOT include chart defaults from `values.yaml`.

`helm get values --all`: shows the COMPLETE merged values including chart defaults from `values.yaml` + all user overrides. This is the full set of values that were used to render templates for the current revision.

Q45. How do you install a chart with a custom values file AND override one additional value?

```
helm install myapp ./mychart -f custom-values.yaml --set image.tag=v2.0.0
```

`--set` has higher priority than `-f`, so `image.tag=v2.0.0` overrides whatever is in `custom-values.yaml`. Multiple `-f` files are applied left to right, with later files taking precedence.

Q46. How do you upgrade a release and ensure it rolls back automatically on failure?

Use `--atomic` flag: `helm upgrade myapp ./mychart --atomic --timeout 10m`

`--atomic` combines `--wait` (wait for all resources to be Ready) with automatic rollback if the wait times

out or any resource fails. It's the standard production upgrade pattern. Without `--atomic`, a failed upgrade leaves the release in 'failed' state that must be manually resolved.

Q47. How do you render a chart's templates locally without a cluster?

```
helm template [flags]
```

Example: `helm template my-release ./mychart -f values.yaml > rendered.yaml`

No cluster connection needed. Useful for CI pipeline linting, debugging template rendering, and validating YAML syntax. Use `--debug` for verbose output showing source template filenames. Pipe to `kubectl apply --dry-run=client -f -` to test against a local/remote cluster without creating resources.

Q48. How do you install a specific version of a chart?

```
helm install myapp bitnami/nginx --version 15.1.0
```

For upgrades: `helm upgrade myapp bitnami/nginx --version 15.2.0`

First ensure the repo is updated: `helm repo update bitnami`. To find available versions: `helm search repo bitnami/nginx --versions`

Q49. How do you pass a multi-line string as a `--set` value?

Use `\n` for newlines in `--set`:

```
helm install myapp ./chart --set 'config=line1\nline2\nline3'
```

Better approach for multi-line content: use `--set-file` to read from a file:

```
helm install myapp ./chart --set-file config=./my-config.txt
```

Or use a values file (`-f`) with proper YAML block scalar:

```
# values-override.yaml:
```

```
config: |
```

```
line1
```

```
line2
```

```
line3
```

```
helm install myapp ./chart -f values-override.yaml
```

Q50. How do you see what manifest a currently deployed release actually contains?

```
helm get manifest -n
```

This retrieves the rendered YAML manifests that were applied during the last successful deployment from the stored release Secret. It shows exactly what Helm deployed, which may differ from what `kubectl` currently shows if resources were manually modified after deployment.

Q51. What does helm history output? Explain the STATUS column values.

helm history shows: REVISION (number), UPDATED (timestamp), STATUS, CHART (chart+version), APP VERSION, DESCRIPTION.

STATUS values:

deployed — this revision is the current active one.

failed — this revision's operation failed.

superseded — replaced by a newer revision.

uninstalled — removed via helm uninstall --keep-history.

pending-install/upgrade/rollback — operation in progress or stuck.

Q52. How do you rollback to revision 2 of a release?

```
helm rollback 2 -n
```

To rollback to the previous revision (without specifying number):

```
helm rollback -n
```

To rollback to the last DEPLOYED revision (skip failed ones): use revision 0:

```
helm rollback 0 -n
```

Q53. How do you uninstall a release but keep its history?

```
helm uninstall -n --keep-history
```

With --keep-history, the release Secrets are NOT deleted. The release status becomes 'uninstalled'. You can later helm install with the same name and helm will recognize the existing history. Without --keep-history (default), all release Secrets are deleted and history is lost.

Q54. How do you install a chart with a generated release name?

```
helm install --generate-name bitnami/nginx
```

Helm generates a unique name based on the chart name + random suffix, e.g., 'nginx-1699547823'.

Customize the name template:

```
helm install --generate-name --name-template 'myapp-{{randAlpha 6 | lower}}' bitnami/nginx
```

Q55. How do you search for all versions of a specific chart in a repository?

```
helm search repo bitnami/nginx --versions
```

This lists ALL available versions of bitnami/nginx in the local repo cache. Without --versions, only the latest version is shown.

To find a specific version: `helm search repo bitnami/nginx --version '~15.1.0'`

SECTION E: Hooks Questions (Q56–Q62)

Q56. What are Helm hooks? List all hook types.

Hooks are Kubernetes resources (Jobs, Pods, etc.) annotated with `helm.sh/hook` that Helm executes at specific lifecycle points. They run to completion before Helm proceeds.

All hook types:

pre-install, post-install

pre-upgrade, post-upgrade

pre-rollback, post-rollback

pre-delete, post-delete

test (runs only when 'helm test' is explicitly executed)

Q57. How do you define a Kubernetes Job as a pre-upgrade hook?

Add the hook annotation to the Job's metadata:

annotations:

'helm.sh/hook': pre-upgrade

'helm.sh/hook-weight': '-5'

'helm.sh/hook-delete-policy': before-hook-creation, hook-succeeded

The Job must have `spec.template.spec.restartPolicy`: `Never` or `OnFailure` (not `Always`, as Jobs require finite completion).

Q58. What is `helm.sh/hook-weight`? How does it control hook execution order?

`hook-weight` is a string integer annotation that determines execution order when multiple hooks of the same type exist. Hooks with LOWER (more negative) weights run FIRST.

Example: weight `'-10'` runs before `'-5'` which runs before `'0'` which runs before `'10'`.

Default weight is 0 if not specified. Hooks with equal weight run in alphabetical order by resource name. The weight MUST be a string (`'-5'`, not `-5`).

Q59. What is `helm.sh/hook-delete-policy`? Explain each possible value.

`before-hook-creation`: Delete the hook resource BEFORE creating a new one of the same name. Prevents accumulation of old job pods. This is the DEFAULT.

`hook-succeeded`: Delete the hook resource AFTER it completes successfully. Keeps failed resources for debugging.

hook-failed: Delete the hook resource AFTER it fails. Automatically cleans up failed jobs.

Multiple policies can be combined comma-separated: 'before-hook-creation,hook-succeeded'

Q60. What happens to a release if a hook fails?

If a hook Job exits with non-zero, exceeds its backoffLimit, or exceeds the hook timeout:

1. Helm marks the entire release as 'failed'.
2. No further hooks or chart resources are processed.
3. The release is NOT automatically rolled back (unless --atomic was used).
4. The hook resource remains in the cluster (unless hook-failed delete policy is set).

To recover: delete the failed hook resource manually, then retry the helm operation.

Q61. How do you write a Helm test? What annotation does a test pod need?

A Helm test is a Pod (or Job) in templates/tests/ with the annotation helm.sh/hook: test.

templates/tests/test-connection.yaml:

apiVersion: v1

kind: Pod

metadata:

name: {{ include 'mychart.fullname' . }}-test

annotations:

'helm.sh/hook': test

'helm.sh/hook-delete-policy': before-hook-creation,hook-succeeded

spec:

containers:

- name: test

image: busybox

command: ['sh', '-c', 'wget -q -O- http://{{ include 'mychart.fullname' . }}:{{ .Values.service.port }}']

restartPolicy: Never

Run with: helm test -n

Q62. What is the difference between a hook and a regular template resource?

Regular template resources: created/updated/deleted as part of the release lifecycle. Included in helm get manifest. Tracked by Helm as part of the release.

Hook resources: executed at specific lifecycle points, run to completion, then optionally deleted. NOT tracked as ongoing release resources. NOT included in helm get manifest (use helm get hooks). Helm waits for them to complete (success/failure) before proceeding. If a hook resource is not deleted (no

delete policy), it persists in the namespace but is not managed by subsequent upgrades.

SECTION F: Security Questions (Q63–Q68)

Q63. How do you read a Helm release secret with kubectl?

```
kubectl get secrets -n --field-selector type=helm.sh/release.v1
```

Lists all Helm release secrets

Decode a specific revision:

```
kubectl get secret sh.helm.release.v1..v -n \
```

```
-o jsonpath='{.data.release}' | base64 -d | base64 -d | gunzip | python3 -m json.tool
```

The double base64 -d is required because K8s Secrets are base64-encoded AND Helm additionally base64-encodes the JSON payload before storing it.

Q64. Why is --set password=mypassword insecure? What are secure alternatives?

--set values are insecure because: (1) Shell history (~/.bash_history) records the full command. (2) Process list (ps aux) shows running commands. (3) The value is stored in plain base64 in the release Secret — readable by anyone with get secrets permission. (4) CI/CD logs may capture the full command.

Secure alternatives:

- (1) Sealed Secrets: encrypt the secret with the cluster's public key, safe to commit to git.
- (2) helm-secrets + SOPS: encrypt values files with AWS KMS, GCP KMS, or PGP key.
- (3) External Secrets Operator: sync from AWS Secrets Manager, HashiCorp Vault, etc.
- (4) Create the K8s Secret separately (before helm install) and reference it by name in values.

Q65. What is the Helm Secrets plugin? How does it work with Mozilla SOPS?

helm-secrets is a Helm plugin (helm plugin install <https://github.com/jkroepke/helm-secrets>) that transparently encrypts/decrypts values files using Mozilla SOPS.

Workflow:

1. helm secrets enc secrets.yaml — encrypts with KMS/PGP key
2. Commit encrypted secrets.yaml to git (safe)
3. helm secrets install myapp ./chart -f secrets.yaml — decrypts at deploy time
4. SOPS supports AWS KMS, GCP KMS, Azure Key Vault, and PGP as key backends
5. Encrypted file is only decryptable by identities authorized in the .sops.yaml config

Q66. What RBAC permissions does a service account need to perform helm install?

Minimum permissions required:

- Secrets: get, list, create, update, delete (for Helm release storage)
- All resource types the chart creates (Deployments, Services, ConfigMaps, ServiceAccounts, etc.)

If the chart creates RBAC resources or CRDs: ClusterRole permissions needed.

Create a Role/RoleBinding scoped to the target namespace:

```
apiGroups: ['','apps','networking.k8s.io']
resources: ['secrets','deployments','services','configmaps','serviceaccounts','ingresses']
verbs: ['get','list','create','update','patch','delete']
```

Q67. How do you sign a Helm chart? What is a provenance file?

Sign with GPG:

```
helm package ./mychart --sign --key 'Your GPG Key Name' --keyring ~/.gnupg/secring.gpg
```

This creates two files: mychart-1.0.0.tgz (the chart archive) and mychart-1.0.0.tgz.prov (provenance file).

The .prov file contains: (1) The chart metadata (from Chart.yaml). (2) SHA-256 hash of the .tgz archive. (3) PGP signature of the hash using the signer's private key. Recipients can verify the signature with the signer's public key to confirm the chart wasn't tampered with.

Q68. How do you verify a signed chart with helm verify?

```
helm verify mychart-1.0.0.tgz --keyring ~/.gnupg/pubring.gpg
```

This: (1) Checks that the .prov file exists alongside the .tgz. (2) Verifies the PGP signature in .prov using the provided keyring. (3) Recomputes the SHA-256 hash of the .tgz and compares with the hash in .prov.

To install only if verified:

```
helm install myapp mychart-1.0.0.tgz --verify --keyring ~/.gnupg/pubring.gpg
```

If verification fails, installation is aborted.

SECTION G: Dependencies Questions (Q69–Q73)

Q69. How do you add a dependency to a chart? Show the Chart.yaml syntax.

In Chart.yaml:

```
dependencies:
- name: postgresql
  version: '12.x.x' # SemVer constraint
  repository: 'https://charts.bitnami.com/bitnami'
  condition: postgresql.enabled # Optional: enable/disable flag
```

alias: db # Optional: rename sub-chart

Then run: `helm dependency update`

This downloads postgresql to charts/ and creates Chart.lock.

Q70. What is the difference between helm dependency update and helm dependency build?

`helm dependency update`: Re-resolves all dependencies from Chart.yaml constraints. Downloads the latest matching versions from repositories. Updates Chart.lock with new exact versions. Requires internet access and configured repositories.

`helm dependency build`: Downloads EXACTLY the versions pinned in Chart.lock. Does NOT re-resolve. Reproducible — always gets the same versions. Use in CI/CD for deterministic builds. Fails if Chart.lock is missing (run update first).

Q71. How do you enable/disable a sub-chart dependency using a condition?

In Chart.yaml dependency:

condition: postgresql.enabled

In values.yaml:

postgresql:

enabled: true # set to false to disable the sub-chart

To disable at install time:

`helm install myapp ./chart --set postgresql.enabled=false`

When disabled, the sub-chart's templates are not rendered and no resources are created.

Q72. How do you pass values to a sub-chart from the parent?

In parent values.yaml, use the sub-chart name (or alias) as the top-level key:

For sub-chart named 'postgresql':

postgresql:

auth:

password: 'mysecret'

primary:

persistence:

size: 10Gi

The postgresql sub-chart accesses these as .Values.auth.password, .Values.primary.persistence.size

For aliased sub-charts, use the alias name:

```
# alias: db in Chart.yaml
```

```
db:
```

```
auth:
```

```
password: 'mysecret'
```

Q73. What are global values and how do they work across chart dependencies?

A special 'global' key in values.yaml is automatically accessible to ALL sub-charts (and the parent chart) as `.Values.global.*`:

Parent values.yaml:

```
global:
```

```
imageRegistry: 'harbor.example.com'
```

```
imagePullSecrets: [{name: regcred}]
```

In any sub-chart template:

```
image: {{ .Values.global.imageRegistry }}/{{ .Values.image.repository }}:{{ .Values.image.tag }}
```

Global values enable cross-chart configuration without requiring each sub-chart to accept custom parameters. Commonly used for: registry overrides, pull secrets, storage class, environment name.

SECTION H: Repository Questions (Q74–Q78)

Q74. How do you add the Bitnami Helm repository?

```
helm repo add bitnami https://charts.bitnami.com/bitnami
```

```
helm repo update
```

Verify: `helm repo list`

Search: `helm search repo bitnami/nginx`

Q75. How do you search for an nginx chart across all public repositories?

Search Artifact Hub (global search, no local config needed):

```
helm search hub nginx
```

Search locally configured repos:

```
helm repo update # fetch latest indexes
```

```
helm search repo nginx
```

Q76. How do you pull a chart to inspect it locally without installing?

```
helm pull bitnami/nginx
```

```
# Downloads nginx-.tgz to current directory
```

```
helm pull bitnami/nginx --untar
# Downloads and extracts to ./nginx/ directory
```

```
helm pull bitnami/nginx --version 15.1.0
# Pull specific version
```

Helm show without downloading:

```
helm show values bitnami/nginx # just values.yaml
helm show chart bitnami/nginx # just Chart.yaml
helm show all bitnami/nginx # everything
```

Q77. How do you create and host your own Helm chart repository on GitHub Pages?

1. Package charts: `helm package ./mychart`
2. Create index: `helm repo index . --url https://.github.io/`
3. Push chart .tgz files + index.yaml to gh-pages branch
4. Add to helm: `helm repo add myrepo https://.github.io/`

Automate with GitHub Actions: on push to main, helm package, helm repo index, commit to gh-pages branch.

Q78. What is helm repo update and when should you run it?

`helm repo update` fetches the latest index.yaml from all configured repositories and updates the local cache. Without it, `helm search repo` and `helm install` use stale data and may install outdated chart versions.

Run when: before installing a chart, to get the latest versions; in CI/CD before any helm install/upgrade; before doing `helm search repo` to get accurate version information. By default Helm does NOT auto-update the cache.

SECTION I: CI/CD & Production Questions (Q79–Q85)

Q79. Describe a production-safe Helm upgrade pipeline.

Order:

1. `helm lint ./chart --strict -f values.yaml` → catch syntax errors
2. `helm template release ./chart -f values.yaml > manifest.yaml` → render
3. `kubeconform manifest.yaml` → validate against K8s schemas
4. `conftest test manifest.yaml` → enforce OPA policies
5. `helm diff upgrade release ./chart -f values.yaml` → review changes
6. (manual approval gate for production)
7. `helm upgrade --install release ./chart -f values.yaml --atomic --timeout 10m` → deploy

8. helm test release → smoke test
9. Notify on success/failure

Q80. What is Helmfile? How is it different from running multiple helm install commands?

Helmfile is a declarative configuration file (helmfile.yaml) for managing multiple Helm releases. It defines repos, environments, default values, and releases in a single file.

Differences from multiple helm install: (1) Declarative: helmfile apply is idempotent — installs missing, upgrades changed releases. (2) Environment support: built-in dev/staging/prod environment templating. (3) Dependency ordering: 'needs' field controls release install order. (4) Atomic operations: helmfile apply --interactive for all-or-nothing deploys. (5) diff/sync/destroy commands for lifecycle management.

Q81. How does ArgoCD use Helm? Does ArgoCD run helm install directly?

ArgoCD does NOT run helm install. It runs helm template to render manifests, then applies them with kubectl apply. This means:

- Helm hooks DO NOT run in ArgoCD (hooks are not part of the rendered manifests)
- No helm history (ArgoCD uses its own sync history)
- No NOTES.txt displayed
- ArgoCD tracks sync status against rendered templates, not Helm release status

Flux HelmRelease CRD: uses the Helm SDK directly — does run helm install/upgrade, so hooks work and history is maintained.

Q82. What is the helm diff plugin and why is it essential before production upgrades?

helm diff upgrade shows the DIFF between the currently deployed release and what the new upgrade would apply. It shows added, removed, and changed lines in Kubernetes manifests — like git diff but for Helm releases.

Install: helm plugin install <https://github.com/databus23/helm-diff>

Usage: helm diff upgrade myapp ./chart -f prod-values.yaml

Why essential: (1) Preview exactly what changes before they hit production. (2) Catch unintended changes (e.g., a new chart version changes default resource limits). (3) Required for change management approvals. (4) Detect removed resources that won't be cleaned up by default.

Q83. How do you promote a release from dev to prod with different values?

Pattern: shared chart, environment-specific values files.

Directory structure:

chart/ # shared chart

```
values/  
base.yaml # common values  
dev.yaml # dev overrides  
production.yaml # prod overrides
```

Deploy dev:

```
helm upgrade --install myapp ./chart -f values/base.yaml -f values/dev.yaml -n dev
```

Deploy prod:

```
helm upgrade --install myapp ./chart -f values/base.yaml -f values/production.yaml -n prod
```

Values files cascade: base.yaml provides defaults, env-specific files override.

Q84. What is the recommended way to handle image tags across environments?

Use a specific, pinned image tag in values.yaml (never 'latest'). For CI/CD promotion:

1. CI builds image, tags with git SHA: myapp:abc1234
2. CI updates values/dev.yaml: image.tag: abc1234
3. Deploy to dev: helm upgrade --install myapp ./chart -f values/dev.yaml
4. After testing, update values/production.yaml: image.tag: abc1234
5. Deploy to prod: helm upgrade --install myapp ./chart -f values/production.yaml

This ensures exact same image runs in dev and prod. Never use branch-based tags (latest, main) — unpredictable.

Q85. How do you enforce that all production charts have resource limits defined?

Use OPA/Rego policies with Conftest in the CI pipeline:

```
policies/require-limits.rego:  
package main  
deny[msg] {  
  input.kind == 'Deployment'  
  container := input.spec.template.spec.containers[_]  
  not container.resources.limits  
  msg := sprintf('Container %v missing resource limits', [container.name])  
}
```

In CI: helm template myapp ./chart | conftest test -

Also: use values.schema.json to require resources to be defined:

```
'resources': {'required': ['limits', 'requests']}
```

Or use Kyverno/OPA Gatekeeper admission webhooks to block deployments without limits at the cluster level.

SECTION J: Scenario / Troubleshooting Questions (Q86–Q95)

Q86. You run `helm install myapp ./myapp-chart` and it hangs. How do you diagnose?

The most common cause is `--wait` (or `--atomic`) waiting for pods to become Ready.

Diagnosis steps:

1. Open another terminal: `kubectl get pods -n -w` — watch pod status
2. `kubectl describe pod -n` — look for: `ImagePullBackOff`, `CrashLoopBackOff`, `pending` (no nodes match `nodeSelector/tolerations/resources`)
3. `kubectl get events -n --sort-by='.lastTimestamp'` — check for scheduling failures
4. `kubectl logs -n` — application errors
5. Check services: `kubectl get svc` — `LoadBalancer` pending external IP (common cause of `--wait` hang)

Fix: `Ctrl+C` to interrupt, fix the underlying issue, `helm uninstall myapp`, `retry`.

Q87. You get 'Error: UPGRADE FAILED: another operation in progress'. What caused it?

Cause: A previous upgrade (or install/rollback) was interrupted (network timeout, `Ctrl+C`, pod crash) and left the release in 'pending-upgrade' state. Helm uses the release status as a lock.

Diagnosis: `helm history myapp -n` — look for `pending-upgrade` status.

Fix — Option 1 (recommended): `helm rollback myapp 0 -n`. Revision 0 = last deployed revision. This resets state to last known good and creates a new 'deployed' revision.

Fix — Option 2 (manual): Find and patch the pending Secret to mark it as failed, then `retry`.

Fix — Option 3 (nuclear): `helm uninstall myapp --keep-history`; then `reinstall`.

Q88. Your `helm rollback` fails with 'Error: release has no deployed releases'. What does this mean?

This error means there is no revision in the release history with `STATUS=deployed`. This happens when:

1. Every revision is in 'failed' status (failed installs/upgrades)
2. The release was uninstalled without `--keep-history` and partially exists

Recovery:

1. `helm history myapp` — check all revisions and their statuses

2. Try rolling back to the highest revision number explicitly: `helm rollback myapp`
3. If all revisions are failed: `helm uninstall myapp (force cleanup)`, then `helm install fresh`
4. Recover from backup if data was involved

Q89. A hook job is failing and blocking your release. How do you clean it up and proceed?

1. Identify the stuck hook: `kubectl get jobs -n -l heritage=Helm`
2. Check logs: `kubectl logs -n job/`
3. Delete the failed job: `kubectl delete job -n`
4. Fix the underlying issue (database connectivity, permissions, etc.)
5. Retry the helm operation: `helm upgrade myapp ./chart -n`

Preventive measures:

- Set `helm.sh/hook-delete-policy: before-hook-creation` on all hooks
- Use `--atomic` to automatically clean up on failure
- Set appropriate `activeDeadlineSeconds` on hook Jobs

Q90. You need to see exactly what YAML Helm will apply before running helm upgrade.

```
helm diff upgrade myapp ./chart -f values.yaml -n production
# Shows diff between current deployed state and new chart (requires helm-diff plugin)
```

OR without the plugin:

```
helm template my-release ./chart -f values.yaml > new-manifest.yaml
helm get manifest myapp -n production > current-manifest.yaml
diff current-manifest.yaml new-manifest.yaml
```

OR use `--dry-run`:

```
helm upgrade myapp ./chart -f values.yaml --dry-run --debug 2>&1 | grep -A 1000 'MANIFEST'
```

Q91. A colleague deleted a K8s deployment that was part of a Helm release. helm status shows 'deployed'. How do you reconcile?

`helm status` says 'deployed' because Helm's release Secret still records the last operation as successful. Helm doesn't continuously monitor cluster state.

Reconcile by running `helm upgrade` (which re-applies all manifests):

```
helm upgrade myapp ./chart -f values.yaml -n production
```

Helm's 3-way merge detects that the Deployment is missing from live state and re-creates it.

Alternatively: `helm upgrade --install --force myapp ./chart -f values.yaml`

For continuous reconciliation (detect and fix drift automatically), use ArgoCD or Flux instead of raw

Helm.

Q92. You accidentally ran helm uninstall on a production release. What options do you have to recover?

Options (best to worst):

1. IF --keep-history was used: the release Secrets still exist. Run helm install with the same name — Helm recognizes existing history. Then rollback to the desired revision.
2. IF release Secrets were deleted: run helm get manifest (fails) and check if you have rendered manifests in your CI/CD pipeline artifacts. Re-apply with kubectl apply -f.
3. Recover from GitOps source: if using GitOps, re-sync from the Git repo — ArgoCD/Flux will recreate everything.
4. Recover from backup: restore PVC data from backup, then reinstall chart.
5. Lesson learned: set HELM_MAX_HISTORY and use --keep-history in production; implement GitOps for continuous reconciliation.

Q93. A chart template fails with 'nil pointer evaluating interface {}.tag'. How do you fix this?

Error location: .Values.image.tag — the parent key .Values.image is nil (not set in values), so accessing .tag panics.

Fixes:

1. Ensure values.yaml has the key:

image:

tag: 'alpine'

2. Use 'with' to guard nested access:

```
{{- with .Values.image }}
```

```
image: '{{ .repository }}:{{ .tag }}
```

```
{{- end }}
```

3. Use 'default' with full path:

```
image: '{{ .Values.image.repository | default 'nginx' }}:{{ .Values.image.tag | default 'latest' }}
```

4. Check with index function:

```
{{ $tag := index .Values.image 'tag' | default 'latest' }}
```

Best practice: always ensure all nested paths in values.yaml have defaults.

Q94. You want to install the same chart twice in the same namespace with different configurations.

Give each installation a different release name and use different values:

```
helm install app-blue ./mychart \n --set color=blue --set replicaCount=3 -n production
```

```
helm install app-green ./mychart \n --set color=green --set replicaCount=1 -n production
```

IMPORTANT: The chart must use `{{ include 'chart.fullname' . }}` for all resource names (includes `Release.Name`), not hardcoded names. Otherwise both installs create resources with the same name and conflict.

This pattern is used for: blue-green deployments, A/B testing, canary releases, multi-tenant installations.

Q95. Your chart works in dev but fails in prod because prod doesn't have a CRD. How do you handle this gracefully?

Use `.Capabilities.APIVersions.Has` to conditionally render CRD-dependent resources:

```
{{- if .Capabilities.APIVersions.Has 'cert-manager.io/v1/Certificate' }}
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
name: {{ include 'mychart.fullname' . }}-tls
spec:
# ...
{{- end }}
```

Add a `values.yaml` toggle:

```
certManager:
enabled: true # set false in environments without cert-manager
```

```
{{- if .Values.certManager.enabled and .Capabilities.APIVersions.Has
'cert-manager.io/v1/Certificate' }}
```

Also: move CRD-dependent resources into a sub-chart with a condition, so they can be disabled entirely for clusters without the CRD. Document the CRD requirement in `NOTES.txt` and `README`.

PART 7: QUICK REFERENCE

Chapter 20 — Helm Cheat Sheet

All Helm Commands — Quick Reference

Command	Description
helm install	Install a release
helm install --atomic	Install with wait + auto-rollback
helm install --dry-run	Simulate install (hits API server)
helm install --generate-name	Auto-generate release name
helm install -f vals.yaml	Install with custom values file
helm install --set k=v	Install with inline value override
helm upgrade	Upgrade a release
helm upgrade --install	Upgrade or install if not exists
helm upgrade --atomic	Upgrade with wait + auto-rollback
helm upgrade --reset-values	Discard previous values on upgrade
helm upgrade --reuse-values	Keep previous values on upgrade
helm list	List releases in current namespace
helm list -A	List releases in all namespaces
helm list --all	Include failed/pending releases
helm status	Show release status and NOTES
helm get manifest	Show deployed K8s manifests
helm get values	Show user-supplied values
helm get values --all	Show all values (defaults + user)
helm get hooks	Show hook manifests
helm get notes	Show NOTES.txt output
helm get all	Show everything
helm history	Show release revision history
helm rollback	Rollback to specific revision
helm rollback 0	Rollback to last deployed revision

Command	Description
helm uninstall	Uninstall release (delete all resources)
helm uninstall --keep-history	Uninstall but keep history
helm template	Render templates locally (no cluster)
helm lint	Check chart for errors
helm lint --strict	Fail on warnings too
helm test	Run test hooks
helm repo add	Add a repository
helm repo list	List repositories
helm repo update	Fetch latest repo indexes
helm repo remove	Remove a repository
helm search repo	Search configured repos
helm search hub	Search Artifact Hub
helm show chart	Show Chart.yaml metadata
helm show values	Show default values.yaml
helm show all	Show all chart info
helm dependency update	Download deps, update Chart.lock
helm dependency build	Download deps from Chart.lock
helm dependency list	List resolved dependencies
helm package	Create .tgz chart archive
helm package --sign --key 'K'	Create signed chart with .prov
helm verify	Verify chart signature
helm repo index	Generate repository index.yaml
helm registry login	Authenticate to OCI registry
helm push oci://url	Push chart to OCI registry
helm pull oci://url --version v	Pull chart from OCI registry
helm version	Show Helm version

Command	Description
helm env	Show all HELM_* env vars
helm plugin install	Install a plugin
helm plugin list	List plugins
helm plugin update	Update a plugin
helm completion bash	Generate bash completion script

HELM_* Environment Variables Quick Reference

Variable	One-Line Description
HELM_CACHE_HOME	Local chart cache directory (repos & downloaded charts)
HELM_CONFIG_HOME	Config directory (repositories.yaml, registry auth)
HELM_DATA_HOME	Data directory (plugins, starters)
HELM_DEBUG	Set 1 for verbose debug output
HELM_DRIVER	Release storage: secret (default) configmap memory sql
HELM_KUBEAPISERVER	Override K8s API server URL
HELM_KUBECONTEXT	Override kubeconfig context
HELM_KUBETOKEN	Bearer token for K8s API auth (CI/CD)
HELM_KUBEINSECURE_SKIP_TLS_VERIFY	Skip TLS verification (dev only!)
HELM_NAMESPACE	Default namespace for commands
HELM_MAX_HISTORY	Max revision history per release (default 10)
HELM_PLUGINS	Override plugins directory
HELM_REGISTRY_CONFIG	OCI registry auth config path
HELM_REPOSITORY_CACHE	Override chart repo cache path
HELM_REPOSITORY_CONFIG	Override repositories.yaml path

Built-in Template Objects Quick Reference

Object.Field	Value
.Release.Name	Release name (e.g., my-nginx)
.Release.Namespace	Target namespace
.Release.IsInstall	true on initial install
.Release.IsUpgrade	true on upgrade/rollback
.Release.Revision	Integer revision number (1 on install)
.Release.Service	Always 'Helm'
.Chart.Name	Chart name from Chart.yaml
.Chart.Version	Chart version (SemVer)
.Chart.AppVersion	Application version (informational)
.Values.*	User + default values (merged)
.Capabilities.KubeVersion.GitVersion	Full K8s version string
.Capabilities.APIVersions.Has 'a/v/K'	Check if API group/version/kind exists
.Files.Get 'path'	Read a non-template file from chart
.Template.Name	Current template file path

Most-Used Template Functions Quick Reference

Function	Example
default	{{ .Values.tag default "latest" }}
required	{{ required "tag required" .Values.tag }}
quote	{{ .Values.name quote }} → "my-name"
upper / lower	{{ .Values.env upper }} → PRODUCTION
trim	{{ .Values.name trim }}
replace	{{ .Chart.Version replace "+" "_" }}
contains	{{ if contains "nginx" .Values.image.repository }}
toYaml + nindent	{{ .Values.resources toYaml nindent 2 }}
include	{{ include "chart.labels" . nindent 4 }}

Function	Example
tpl	<code>{{ tpl .Values.configTemplate . }}</code>
lookup	<code>{{ (lookup "v1" "Secret" "ns" "name").data }}</code>
b64enc	<code>{{ .Values.password b64enc }}</code>
sha256sum	<code>{{ .Values.config toYaml sha256sum }}</code>
semverCompare	<code>{{ if semverCompare ">=1.19" .Capabilities.KubeVersion.GitVersion }}</code>
randAlphaNum	<code>{{ randAlphaNum 32 }}</code> → random 32-char string

Hook Types and Delete Policies Reference

Hook Type	Runs When	Delete Policy	Effect
pre-install	Before chart resources created	before-hook-creation	Delete old hook before creating new one (default)
post-install	After all resources Ready	hook-succeeded	Delete after successful completion
pre-upgrade	Before upgrade applied	hook-failed	Delete after failure
post-upgrade	After upgrade complete	— (none)	Keep forever (for debugging)
pre-delete	Before uninstall begins		
post-delete	After all resources deleted		
pre-rollback	Before rollback begins		
post-rollback	After rollback complete		
test	Only when helm test run		

Values Precedence Order (Lowest → Highest)

1. (LOWEST) Chart defaults — values.yaml bundled with chart
2. Sub-chart values.yaml defaults
3. Parent chart overrides for sub-chart keys
4. User values files via -f flag (multiple files: left-to-right, right wins)
5. (HIGHEST) --set flags — always override everything

Chart Directory Structure at a Glance


```

mychart/
  Chart.yaml           # Metadata: name, version, appVersion, description, dependencies
  values.yaml          # Default configuration values
  values.schema.json   # (Optional) JSON Schema to validate values
  charts/              # Sub-chart archives (.tgz) – generated by helm dep update
  crds/                # CRD manifests – applied FIRST before templates
  templates/           # Go template files → rendered to K8s manifests
  ■   NOTES.txt        # Post-install instructions (printed, not deployed)
  ■   _helpers.tpl      # Named templates (define/include) – NOT rendered
  ■   deployment.yaml
  ■   service.yaml
  ■   ingress.yaml
  ■   serviceaccount.yaml
  ■   hpa.yaml
  ■   tests/
  ■       test-connection.yaml # helm.sh/hook: test
  .helmignore          # Files excluded from helm package

```

Common Error Messages and Fixes

Error Message	Cause	Fix
Error: cannot re-use a name that is still in use	Release with this name already exists	Use helm upgrade --install, or helm uninstall first
Error: UPGRADE FAILED: another operation is in progress	Previous operation left pending state	helm rollback 0
Error: release has no deployed releases	No successful revision to rollback to	helm uninstall + reinstall
nil pointer evaluating interface {}.tag	Parent key nil (e.g., .Values.image missing)	Add key to values.yaml; use 'with' guard; use 'default'
Error: execution error at .../template.yaml: value is required	required function triggered — field is empty	Supply the required value via --set or -f
Error: INSTALLATION FAILED: timed out	--wait/--atomic timeout exceeded	Increase --timeout; debug pod scheduling issues
Error: rendered manifests contain a resource that already exists	Resource exists outside Helm control	helm upgrade --force; or delete the orphan resource
Error: no repositories configured	helm repo add not run	helm repo add ; helm repo update
cannot unmarshal !!str into int32	Wrong YAML type in values (string vs int)	Use --set-string; or quote the value in values.yaml
Error: 1 chart(s) linted, 1 chart(s) failed	helm lint found errors	Read lint output; fix template/Chart.yaml issues

■ **EXAM TIP:** FINAL EXAM TIP: The three most tested Helm scenarios in CKA/CKAD are: (1) helm upgrade --install with --atomic and custom values files, (2) helm rollback to a specific revision, and (3) reading helm history output. Practice these until they are muscle memory.